

SRS - Artisync v1.3.0

Repositorio: <https://github.com/JohanCarvajal04/Proyecto-WEB-ARTISYNC>

Índice

Especificación de Requisitos de Software (SRS)	3
Artisync — Plataforma web de comisiones y venta de contenido digital para creadores	3
1. Introducción	4
1.1 Propósito	4
1.2 Alcance	4
1.3 Definiciones, acrónimos y abreviaturas	4
1.4 Referencias	4
1.5 Resumen del documento	5
1.6 Estados del dominio y transiciones (v1.3.0)	5
1.7 Correspondencia con el Anexo C de ISO/IEC/IEEE 29148:2018	6
2. Descripción global	7
2.1 Perspectiva del producto	7
2.2 Funciones del producto (resumen)	7
2.3 Características de los usuarios	7
2.4 Restricciones	7
2.5 Supuestos y dependencias	7
2.6 Matriz de permisos por rol (v1.3.0)	8
2.7 Interfaces externas	8
2.8 Capacidades fuera de alcance	9
3. Requisitos específicos — Funcionales	10
3.0 Historias de Usuario y Criterio INVEST	10
Módulo Seguridad y Control de Acceso	11
Módulo Perfiles, Verificación y Portafolio	12
Módulo Catálogo Dinámico de Servicios	13
Módulo Comunicación y Notificaciones	13
Módulo Legal, Entregables y Finanzas	14
Módulo Social, Comunidad y Sorteos	16
3.1 Requisitos adicionales (post v1.0.0)	16
Módulo Financiero — Retiros y Auditoría de Pagos	16
Módulo de Reportes	17
Módulo de Notificaciones	17
Módulo Catálogo — Gestión Auxiliar	18
Módulo Social — Reseñas	18
Módulo Seguridad — Administración de Cuentas	18
Módulo Auxiliar — Catálogo de Países y Portafolio Social	18
Módulo Portafolio — Gestión de Obras (v1.3.0)	19
Módulo Comunicación — Administración de Infracciones (v1.3.0)	19
4. Requisitos específicos — No funcionales	19
4.1 Requisitos no funcionales adicionales (post v1.0.0)	23
4.2 Requisitos no funcionales adicionales (v1.3.0)	24
5. Matriz de trazabilidad (resumen)	27

6. Evolución de requisitos desde la Entrega 1A	27
7. Métricas de calidad del corpus de requisitos	28
7.1 Volumen y distribución	28
7.2 Estado de verificación	29
7.3 Cobertura de trazabilidad	30
7.4 Estabilidad de requisitos	30
8. Aprobación	32

Especificación de Requisitos de Software (SRS)

Artisync — Plataforma web de comisiones y venta de contenido digital para creadores

- Universidad Técnica Estatal de Quevedo (UTEQ)
- Facultad de Ciencias de la Computación y Diseño Digital · Carrera de Ingeniería de Software
- Proyecto de Fin de Curso (PFC) — Aplicaciones Web, Quinto Nivel · Período Académico Presencial 2026-2027

Integrante	ORCID	Correo institucional
Bone Arroyo, Niurca Scarleth	0009-0002-2219-2800	nbonea@uteq.edu.ec
Carvajal Loor, Johan Stalin	0009-0008-9229-382X	jcarvajall@uteq.edu.ec
Figueroa Morales, Bryan Javier	0009-0009-6357-4996	bfigueroam@uteq.edu.ec
Rios Cuyabazo, Jhon Kevin	0009-0003-7446-9450	(correo institucional pendiente de confirmar — ver nota)

Nota sobre Bone Arroyo, Niurca Scarleth: colabora con el equipo por ser compañera de curso en Administración de Bases de Datos, materia en la que Artisync también se usa como caso de estudio; su aporte se concentra en la capa de base de datos (**db/procs/**). Detalle completo en **CONTRIBUTORS.md**.

El correo institucional de Rios Cuyabazo, Jhon Kevin no está confirmado en ningún artefacto versionado del repositorio (su historial de commits usa correos personales/**noreply**); se deja pendiente en vez de inventarlo — actualizar esta fila cuando se confirme.

- **Docente-director:** Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D. — gguerrero@uteq.edu.ec
- **Representante del equipo:** Johan Stalin Carvajal Loor — jcarvajall@uteq.edu.ec (firma en la sección 8)
- **DOI del software (Zenodo):** 10.5281/zenodo.21978572 — archivado sobre el tag v1.0.0; el software sigue evolucionando (v1.3.0 actual) sin una nueva versión archivada todavía
- **DOI del dataset de mediciones (Zenodo):** 10.5281/zenodo.22236251
- **Repositorio:** <https://github.com/JohanCarvajal04/Proyecto-WEB-ARTISYNC>
- **Conforme a:** ISO/IEC/IEEE 29148:2018 (estructura SRS) · INCOSE Guide to Writing Requirements v4 (calidad de requisitos C1–C15)
- **Versión:** v1.3.0 — Entrega Final
- **Fecha:** 2026-09-08

- **Precede a:** (Versión final)
- **Actualiza a:** SRS v1.2.0 (2026-09-07) — ver historial completo de versiones en `docs/requisitos/CHANGELOG-REQ.md`

Nota de mantenimiento: cada cambio sustantivo respecto a la versión 1A se registra en `docs/requisitos/CHANGELOG-REQ.md`. Los identificadores REQ-F-NNN / REQ-NF-NNN reemplazan a los códigos RF-NN / RNF-NN de la Entrega 1A; la tabla de equivalencia está en la sección 6.

Fuente de verdad del estado. El campo Estado de cada requisito es un espejo de la columna estado de `docs/trazabilidad/matriz.csv`, que es la fuente autoritativa porque es la única que exige, en la misma fila, el módulo, el endpoint, la prueba automatizada y la evidencia que sostienen ese estado. Este documento es la fuente de verdad del **enunciado**, la **prioridad MoSCoW** y el **criterio de aceptación**.

Los valores admitidos son exactamente **pendiente**, **implementado** y **verificado**; **verificado** exige una prueba automatizada que lo respalde. `scripts/validate-traceability.sh` falla si ambos documentos divergen, si un requisito queda por debajo del estado que exige su prioridad sin figurar en `docs/trazabilidad/excepciones-estado.txt`, o si un **Must** verificado no declara prueba. Ambos archivos se actualizan en el mismo commit.

1. Introducción

1.1 Propósito

Este documento especifica, de manera completa y verificable, los requisitos funcionales y no funcionales del sistema Artisync en su versión estable **v1.3.0**. Sirve como fuente única de verdad para la trazabilidad hacia el código, las pruebas automatizadas y la evidencia empírica exigida en la Entrega Final del PFC.

1.2 Alcance

Artisync es una plataforma web que centraliza la comercialización de servicios y productos digitales ofrecidos por profesionales creativos (ilustradores, músicos, desarrolladores, diseñadores, etc.). El sistema conecta a **Creadores** (vendedores) con **Clientes**, gestionando perfiles, catálogo, mensajería, contratos con firma electrónica, flujo de pedidos, pagos vía PayPal Orders v2 con patrón *escrow*, y funciones sociales (seguidores, comentarios, sorteos). El desarrollo se enmarca en un proyecto académico de 17 semanas; algunas capacidades (firma legalmente vinculante, logística física, recomendaciones avanzadas, reporte manual de contenido) quedan fuera de alcance y se documentan como trabajo futuro — ver §2.8 para el detalle de cada exclusión, su justificación y qué existe hoy como cobertura parcial.

1.3 Definiciones, acrónimos y abreviaturas

- **RBAC:** Role-Based Access Control.
- **JWT:** JSON Web Token (RFC 7519).
- **2FA/TOTP:** Autenticación de doble factor basada en contraseñas de un solo uso (RFC 6238).
- **Escrow:** patrón de depósito en garantía; los fondos del Cliente se retienen hasta la aprobación del entregable.
- **SP:** Procedimiento almacenado (Stored Procedure).
- **MoSCoW:** Must, Should, Could, Won't — técnica de priorización de requisitos.

1.4 Referencias

ISO/IEC/IEEE 29148:2018; INCOSE Guide to Writing Requirements v4; RFC 7519 (JWT); RFC 7807 (Problem Details); RFC 6238 (TOTP); ISO/IEC 25010:2011 (calidad de software); OWASP Top 10:2021; documento interno `docs/requisitos/historico/entrega-1a.pdf` (corpus original de requisitos, semana del 4 de junio de 2026).

1.5 Resumen del documento

La sección 2 describe el producto y sus actores. La sección 3 detalla los requisitos funcionales (REQ-F). La sección 4 detalla los requisitos no funcionales (REQ-NF). La sección 5 presenta la matriz de trazabilidad resumida. La sección 6 documenta la evolución de los requisitos desde la Entrega 1A. La sección 7 reporta métricas de calidad del corpus. Las subsecciones 1.6 y 1.7 (añadidas en v1.3.0) documentan los estados del dominio y la correspondencia de este documento con el Anexo C de ISO/IEC/IEEE 29148:2018. La subsección 2.8 documenta, con el mismo rigor que un requisito incluido, las capacidades que quedan explícitamente fuera de alcance.

1.6 Estados del dominio y transiciones (v1.3.0)

Construido por lectura directa de los enums, constantes y lógica de transición reales del backend — no por inferencia de nombres plausibles. Varias suposiciones razonables resultaron incorrectas al verificarlas (ver notas por entidad); se documenta el resultado real.

Pedido — **sin catálogo fijo de etapas**. A diferencia de lo que podría asumirse, **Pedido** no tiene una columna de estado ni un enum de etapas: la etapa vigente se deriva de la fila más reciente en `historial_estados_pedido` (histórico, solo inserción — `PATCH/DELETE` sobre él responden 403 por REQ-NF-013). Las etapas mismas (`EtapasFlujo.nombreEtapas`) son texto libre, únicas globalmente, y cualquiera que gestione un flujo de trabajo puede crear nombres nuevos (`WorkflowServiceImpl.obtenerOCrearEtapas`); su orden y cuál es la etapa final se definen por `FlujoEtapasConfig.numeroOrden` (entero) y `esEtapasFinal` (booleano, que en la práctica solo se usa para mostrar, no para bloquear — el corte real de “última etapa” es el número de orden más alto). Transición: `OrderServiceImpl.avanzarEtapas` exige que, si la etapa actual tiene `requiereEntregable=true`, exista un `EntregableFinal`, y avanza a la siguiente por `numeroOrden`; sin etapa siguiente, rechaza (“El pedido ya se encuentra en la etapa final”). **No existe ninguna función de cancelar un pedido** — se verificó explícitamente (cero resultados para cualquier método `cancelarPedido` en el código). Por separado existe `PropuestaTerminosPedido`, una entidad distinta con su propio enum real (`PENDIENTE`, `ACEPTADA`, `RECHAZADA`, `CANCELADA`) para negociar precio/fecha antes del pedido — no debe confundirse con las etapas del pedido mismo.

Corrección (v1.3.0): el enunciado de REQ-F-014 decía antes “se cierra al llegar a Entregado o Cancelado”. Verificado contra `ChatServiceImpl.cerrarSala`: su único invocador es `DeliverableServiceImpl.aprobarEntrega` (aprobación del entregable). No existe ninguna ruta de cancelación de pedido que la invoque, porque esa función no existe. El enunciado se corrigió para reflejar solo el comportamiento real.

Solicitud de retiro (`WithdrawalRequestServiceImpl`) — enum real de 5 valores (constantes `String`, no `@Enumerated`): `Pendiente`, `Aprobado`, `Pagado`, `Rechazado`, `Fallido`. Transiciones: `solicitar` (Creador) → `Pendiente`; `aprobar` (Auditor Financiero/Admin) ejecuta el payout de PayPal y bifurca por su resultado: `SUCCESS` → `Pagado`, `PENDING/PROCESSING` → `Aprobado`, cualquier otro resultado o excepción → `Fallido`; `rechazar` (con nota obligatoria) → `Rechazado`; `reintentar` solo aplica desde `Fallido` y repite la misma bifurcación. Nota de diseño: no existe ningún mecanismo (webhook o *poller*) que haga avanzar automáticamente una solicitud `Aprobado` a `Pagado` — depende de una acción administrativa manual adicional.

Pago en garantía (escrow) (`PaymentServiceImpl`, `PagoGarantia.estadoFondos`) — confirmado exactamente 3 valores: `Pendiente` (al crear la orden de PayPal) → `Retenido` (al confirmarse el webhook con firma válida y captura `COMPLETED`) → `Liberado` (al aprobar el Cliente el entregable, sin volver a verificar `estadoFondos`: el guardián real contra doble liberación es el booleano `EntregableFinal.estaLiberado`). **No existe un cuarto estado de cancelación o reembolso** — se confirmó explícitamente (cero resultados para “Reembolso”/“Disputa”/“refund” en el código); es precisamente la brecha que documenta REQ-NF-019.

Entregable (`EntregableFinal.estaLiberado`) — no es un enum, es un booleano de 2 valores. Nace en `false` al subir el entregable; pasa a `true`, de forma irreversible, cuando el Cliente aprueba (`DeliverableServiceImpl.aprobarEntrega`), lo que además libera el escrow y cierra la sala de chat. Antes de `true`, la descarga del archivo limpio (sin marca de agua) está bloqueada.

Certificado / Verificación (`EstadoVerificacion`, sembrado en `V7__verificacion_asistida_ia.sql`) —

4 valores reales, en **mayúsculas**: PENDIENTE, APROBADO, RECHAZADO, REQUIERE_ACLARACION (no “verificado”: ese valor no existe en ningún lugar del código). Transición inicial **subir** → PENDIENTE; el análisis de IA (**analizarConIa**) no cambia este estado, solo registra un veredicto no vinculante aparte (**veredictoIa**); la decisión humana (**registrarDecision**, bajo un candado de fila que impide sobrescribir una decisión ya tomada) mueve a APROBADO/RECHAZADO/REQUIERE_ACLARACION, y el documento se elimina físicamente solo si el nuevo estado es APROBADO o RECHAZADO (REQUIERE_ACLARACION conserva el archivo). Caso límite real: un *scheduler* diario expira certificados PENDIENTE con más de 30 días y borra su archivo, pero **el estado permanece PENDIENTE** — un certificado puede quedar indefinidamente pendiente con su documento ya eliminado.

1.7 Correspondencia con el Anexo C de ISO/IEC/IEEE 29148:2018

Sección de este SRS	Sección equivalente del Anexo C	Nota
§1.1-1.5 Introducción	C.1 Introduction	Completo
§1.6 Estados del dominio	C.2.4 (comportamiento del sistema, modelos de estado)	Añadido en v1.3.0
§2.1-2.5 Descripción global	C.2.1-C.2.3 (perspectiva, funciones, restricciones)	Completo
§2.6 Matriz de permisos por rol	C.2.6 (características de los usuarios / control de acceso)	Añadido en v1.3.0
§2.7 Interfaces externas	C.2.2 (interfaces del sistema)	Añadido en v1.3.0
§2.8 Capacidades fuera de alcance	C.1 (declaración de alcance, incluidas sus exclusiones)	Añadido tras la ronda de revisión externa sobre v1.3.0 — antes era una frase suelta dentro de §1.2
§3-4 Requisitos funcionales y no funcionales	C.3 Specific requirements	Completo
§5 Matriz de trazabilidad	C.4 Verification / trazabilidad	Completo (docs/trazabilidad/matriz.csv)
§6 Evolución de requisitos	(no tiene equivalente directo en el Anexo C; práctica adicional del proyecto)	—
§7 Métricas de calidad	(no tiene equivalente directo; práctica adicional del proyecto, alineada con ISO/IEC 25010)	—
§8 Aprobación	C.1 (aprobación del documento)	Completo salvo firma pendiente (ver §8)
Requisitos lógicos de base de datos	C.3 (interface requirements / data)	Desviación consciente: no se repite aquí; ya están completamente especificados en db/schema.sql y los diagramas ER de docs/basedatos/, y duplicarlos en el SRS crearía dos fuentes de verdad divergentes.
Requisitos de usabilidad detallados (más allá de REQ-NF-004/007/008/017)	C.3 (usability requirements)	Desviación consciente: el detalle vive en docs/mediciones/sus/ y docs/mediciones/lighthouse/, referenciados desde los requisitos correspondientes en vez de duplicarse.

2. Descripción global

2.1 Perspectiva del producto

Artisync es un sistema nuevo, independiente, compuesto por un frontend Angular (SPA), un backend Spring Boot que expone una API REST, una base de datos PostgreSQL, una caché Redis, y tres integraciones externas: PayPal Orders v2 (pagos), un servicio de IA para verificación de documentos, y almacenamiento de objetos compatible con S3.

2.2 Funciones del producto (resumen)

Registro y autenticación con RBAC; verificación de identidad y de certificados profesionales; gestión de perfil y portafolio; publicación de productos/servicios en catálogo con atributos dinámicos; búsqueda y filtrado; mensajería en tiempo real con moderación automática de contacto externo; briefing configurable; generación de contrato HTML/PDF con firma electrónica; flujo de pedido por etapas; pagos con PayPal y patrón escrow; entrega con marca de agua y liberación de fondos; revisiones adicionales facturables; funciones sociales (seguidores, comentarios, sorteos).

2.3 Características de los usuarios

Rol	Descripción
Administrador	Supervisa la plataforma, gestiona cuentas, roles, permisos, categorías y el catálogo de plantillas de contrato.
Moderador	Revisa certificados y categorías de autoservicio, modera servicios/comentarios/mensajes, gestiona infracciones.
Auditor Financiero	Audita pagos en garantía y transacciones, gestiona solicitudes de retiro, exporta reportes financieros y de contratos.
Soporte	Asistencia técnica: consulta y suspende usuarios, revoca sesiones, resuelve tickets de revisión.
Creador de contenido	Publica servicios/productos, gestiona perfil y portafolio, atiende pedidos, organiza sorteos.
Cliente registrado	Contrata servicios, compra productos, sigue creadores, participa en sorteos.
Visitante anónimo	Explora contenido público sin autenticarse.

Nota (v1.3.0): Moderador, Auditor Financiero y Soporte son roles reales, sembrados en `artisync/db/seed.sql` con su propio conjunto de permisos (`rol_permisos`), pero no figuraban en esta tabla en versiones anteriores del SRS — una omisión real de especificación, no una simplificación deliberada. Ver §2.6 para el detalle de qué permiso tiene cada rol, extraído directamente del seed y de las anotaciones `@PreAuthorize` del backend.

2.4 Restricciones

Proyecto académico de 17 semanas; equipo reducido; hosting local durante el desarrollo; integración de pagos limitada al entorno sandbox de PayPal; alcance geográfico inicial: Ecuador (validación de mayoría de edad, moneda USD).

2.5 Supuestos y dependencias

Se asume disponibilidad continua de las APIs externas (PayPal sandbox, servicio de IA, almacenamiento S3-compatible) durante las pruebas. La arquitectura se diseña contemplando extensión futura (multi-idioma, multi-moneda) sin comprometerlas en esta entrega.

2.6 Matriz de permisos por rol (v1.3.0)

Tabla construida por extracción directa de `artisync/db/seed.sql` (el seed que realmente ejecuta `docker-compose.yml` antes de que arranque Spring Boot) y de las migraciones Flyway que lo modifican (V10, V19, V20, V32, V33, V36, V39), cruzada con las anotaciones `@PreAuthorize` del backend — no es una asignación inventada. Se muestran los permisos citados explícitamente en la ronda de revisión externa; el catálogo completo tiene 48 permisos sobre 6 roles reales (no 5: **SOPORTE** es un rol sembrado adicional, ver nota de §2.3).

Permiso	ADMIN	MODERADOR	AUDITOR_FINANCIERO	CREADOR	CLIENTES	SOPORTE
PAGO_AUDITAR	1		Sí			
TRANSACCION_VER	1		Sí			
REPORTE_FINANCIERO_EXPORTAR	1		Sí			
REPORTE_CONTRATO_EXPORTAR	1		Sí			
RETIROS_GESTIONAR	1		Sí			
RETIROS_SOLICITAR				Sí		
CATEGORIA_CREAR				Sí		
CATEGORIA_GESTIONAR	Sí	Sí				
INFRACCION_GESTIONAR	1	Sí				
CERTIFICADO_REVISAR	1	Sí				
SERVICIO_MODERAR	1	Sí				
USUARIO_VER	Sí					Sí
USUARIO_CREAR	Sí					
USUARIO_EDITAR	Sí					
USUARIO_ELIMINAR	Sí					
USUARIO_SUSPENDER	Sí					Sí
USUARIO_EXPORTAR	Sí					Sí
ROL_GESTIONAR	Sí					
SESION_REVOCAR	Sí					Sí
CONTRATO_PLANTILLA_GESTIONAR	Sí					

¹ ADMIN no tiene la fila de permiso en `rol_permisos` — `V32__ajuste_permisos_admin_moderador.sql` la retiró deliberadamente para especializar el rol (comentario propio de la migración) — pero todos los controladores de estas filas están anotados `hasAuthority('X')` or `hasRole('ADMIN')`: el comodín de rol hace que ADMIN pueda llamar el endpoint en la práctica, aunque la fila de permiso ya no exista en la base de datos. La tabla refleja lo que la base de datos concede; para “quién puede llamar el endpoint hoy”, sumar ADMIN a cada fila marcada ¹. Las dos únicas filas sin comodín ADMIN son `RETIROS_SOLICITAR` (acción exclusiva del Creador, ADMIN nunca la tuvo) y `CONTRATO_PLANTILLA_GESTIONAR` (ADMIN la tiene por fila directa, no necesita comodín).

2.7 Interfaces externas

Interfaz	Protocolo / formato	Autenticación	Comportamiento ante indisponibilidad
PayPal Orders v2 (REQ-F-020, REQ-NF-014, REQ-NF-019)	HTTPS REST, JSON; webhooks entrantes firmados	Credenciales OAuth2 de PayPal vía variables de entorno (<code>.env</code>); verificación de firma de webhook (<code>webhook-id</code>)	Sin reconciliación activa hoy (ver REQ-NF-019): si el webhook no llega, el pago queda en Pendiente indefinidamente sin reintento automático — brecha declarada, no comportamiento diseñado
Servicio de IA — verificación de identidad y certificados (REQ-F-006, REQ-F-007)	HTTPS REST; payload = prompt genérico + imagen en base64 (sin nombre, correo ni número de documento como campos separados — ver REQ-NF-018)	Clave de API del proveedor (Gemini/NVIDIA) vía variable de entorno	Sin respuesta o error del proveedor: el certificado permanece en estado PENDIENTE (ver §1.6); no hay reintento automático documentado más allá del scheduler de expiración a 30 días
Almacenamiento de objetos compatible con S3 (Azure Blob Storage) (REQ-NF-011)	HTTPS; SDK de Azure Blob	Cadena de conexión / SAS token vía variable de entorno (<code>documentos.proveedor</code> , <code>DOCUMENTOS_PROVEEDOR</code>)	Proveedor configurable con AlmacenamientoLocal como alternativa de respaldo en código; en producción, mientras <code>DOCUMENTOS_PROVEEDOR</code> no esté fijado a azure en <code>render.yaml</code> , el sistema usa almacenamiento local por defecto — el propio incumplimiento activo declarado en REQ-NF-011, no una estrategia de failover
Canal WebSocket interno (REQ-F-014)	STOMP sobre WebSocket, con <i>fallback</i> a SockJS para navegadores sin soporte nativo (<code>WebSocketConfig.java</code>)	JWT validado por un interceptor STOMP en cada conexión, antes de procesar cualquier mensaje	No es una interfaz con un tercero externo (es interna, entre cliente y backend); ante desconexión, SockJS reintenta la conexión según su propio mecanismo de <i>fallback</i> ; no hay cola de mensajes persistente del lado del servidor para reentrega tras una reconexión

2.8 Capacidades fuera de alcance

Esta sección reemplaza la enumeración suelta que antes vivía dentro de la nota de alcance de §1.2. Cada exclusión se documenta con lo mismo que exige un requisito incluido — qué cubre hoy el sistema, qué no

cubre, y por qué se dejó fuera —, para que la ausencia de una capacidad sea una decisión trazable y no una omisión silenciosa.

Reporte manual de contenido. No existe ningún mecanismo por el cual un usuario (Cliente o Creador) pueda señalar contenido ajeno que considera inapropiado, fraudulento o abusivo — se verificó explícitamente contra el código: no hay entidad, endpoint, ni cola de revisión para una denuncia iniciada por un tercero.

- *Qué cubre hoy la plataforma, parcialmente:* REQ-F-015 detecta automáticamente teléfonos/correos dentro de mensajes de chat, bloquea el mensaje antes de persistirse y registra una infracción; REQ-F-033 permite al Administrador (o titular de `INFRACCION_GESTIONAR`) listar esas infracciones, consultar el historial por usuario y revertir manualmente una suspensión.
- *Qué NO cubren esos dos requisitos:* (a) contenido no textual — una imagen de portafolio, una foto de perfil o el archivo de un servicio no pasan por ningún análisis, automático ni manual; (b) una denuncia iniciada por un tercero que observa el contenido de otro usuario — el único disparador de una infracción hoy es la detección automática de contacto directo en el chat, nunca una acción de un usuario distinto; (c) plazos de revisión — al no existir una cola de denuncias, no hay ningún SLA que gestionar ni que incumplir.
- *Justificación de la exclusión:* la restricción de equipo reducido y proyecto académico de 17 semanas, ya declarada en §2.4, no permitió construir la rodaja vertical completa (entidad, servicio, cola de moderación con SLA, endpoint y UI de “reportar”) que esta capacidad exigiría, comparable en tamaño a REQ-F-028 (moderación de categorías) o REQ-F-033 mismos. No es una capacidad que el equipo considere innecesaria para un sistema en producción: queda como trabajo futuro explícito, no como alcance descartado por diseño.
- *Trabajo futuro:* de construirse, seguiría el mismo patrón de REQ-F-028/033 — entidad con estado (**pendiente/revisado/descartado**), endpoint de creación abierto a cualquier usuario autenticado, y endpoints de listado/resolución restringidos a `INFRACCION_GESTIONAR` o al rol Moderador — y se documentaría con su propio identificador **REQ-F-ONN** siguiendo el modelo de rationale + criterio de aceptación + verificación de los requisitos de §3.1.

Otras capacidades fuera de alcance (firma legalmente vinculante, logística física de productos, recomendaciones avanzadas) se mantienen como estaban: declaradas en el alcance del proyecto (§1.2) sin un requisito propio, por no tener implicación legal, financiera o de seguridad comparable a la del reporte de contenido — la moderación de contenido sí la tiene, porque protege a los usuarios entre sí y sostiene la confianza del catálogo público, y por eso es la única exclusión que se documenta con este nivel de detalle.

3. Requisitos específicos — Funcionales

Cada requisito seguido de: **Rationale**, **Prioridad (MoSCoW)**, **Criterio de aceptación**, **Verificación** y **Estado**. Los 23 requisitos originales provienen del corpus de la Entrega 1A y se mantienen con trazabilidad completa (ver tabla de equivalencia en §6; dos de ellos, REQ-F-022 y REQ-NF-001, se dividieron en sub-requisitos atómicos en v1.3.0 — ver §7.4); los 8 requisitos adicionales de §3.1 se incorporaron en v1.1.2/v1.2.0, y REQ-F-032/REQ-F-033 se incorporaron en v1.3.0 — ver `CHANGELOG-REQ.md`.

3.0 Historias de Usuario y Criterio INVEST

Para cada uno de los 23 requisitos funcionales especificados a continuación, se ha documentado su correspondiente historia de usuario en formato Connextra. **La totalidad de las 23 historias de usuario han sido evaluadas rigurosamente y cumplen con los seis atributos del modelo INVEST** (Independent, Negotiable, Valuable, Estimable, Small, Testable). El análisis detallado individual que respalda el cumplimiento de INVEST para cada historia se encuentra archivado en el repositorio bajo `docs/requisitos/historias/`. A modo de ilustración, el análisis documentado para la HU-01 (Registro y autenticación) establece: “*Independiente del resto del flujo de autenticación; negociable en los campos exactos del formulario; valiosa (es la puerta de entrada al sistema); estimable y pequeña (un solo formulario + persistencia); testable mediante el criterio de aceptación.*” Los ocho requisitos funcionales adicionales

de §3.1 (REQ-F-024 a REQ-F-031) tienen igualmente su historia de usuario propia (HU-24 a HU-31, en `docs/requisitos/historias/HU-24-a-HU-31-adicionales.md`), evaluada con el mismo criterio INVEST. Los requisitos no funcionales (REQ-NF) no llevan historia de usuario ni caso de uso propios por diseño: su origen es una norma técnica, un ADR o una decisión de arquitectura, no una necesidad expresada por un rol de usuario — el campo correspondiente en `matriz.csv` permanece vacío intencionalmente para esos casos.

Módulo Seguridad y Control de Acceso

REQ-F-001 (ex RF-01) — El sistema debe permitir el registro de nuevos usuarios con selección de rol (Creador o Cliente); el rol determina las vistas y acciones disponibles durante toda la sesión.

- Rationale: separación de responsabilidades es la base del modelo de negocio de doble lado (oferta/demanda).
- Prioridad: Must
- Aceptación: un Creador no accede a rutas de Cliente y viceversa; el sistema redirige ante acceso no autorizado.
- Verificación: Test (JUnit + MockMvc sobre `AuthController/UserController`)
- Estado: verificado

REQ-F-002 (ex RF-02) — Control de acceso basado en roles (RBAC): cada acción debe estar asociada a un permiso específico asignado al rol.

- Rationale: exigido por RNF de seguridad y por la separación de privilegios del dominio.
- Prioridad: Must
- Aceptación: asignar/revocar un permiso cambia el acceso sin reiniciar sesión (revocación efectiva en la siguiente solicitud).
- Verificación: Test (`RolePermissionControllerTest`)
- Estado: verificado

REQ-F-003 (ex RF-03) — Gestión de sesiones mediante JWT de dos tokens: token de acceso con expiración de 24 horas y token de refresco con expiración de 7 días (configurable vía `JWT_REFRESH_EXPIRATION`); rutas protegidas exigen un token de acceso válido en la cabecera de autorización.

- Rationale: autenticación *stateless* escalable sin sesión en servidor.
- Prioridad: Must
- Aceptación: token válido → 200; token expirado → 401 “Token expirado”; sin cabecera → 401 “Autenticación requerida”; token de sesión cerrada no reutilizable; el token de acceso expira a las 24h y el refresh token a los 7 días (configurable vía `JWT_REFRESH_EXPIRATION`).
- Verificación: Test (`JwtAuthenticationFilterTest`, `AuthServiceImplTest`)
- Estado: verificado

REQ-F-004 (ex RF-04) — Recuperación de contraseña mediante enlace de un solo uso, válido 60 minutos, enviado por correo.

- Rationale: Sin un mecanismo de autoservicio, un usuario que olvida su contraseña dependería de soporte manual; el enlace de un solo uso y su expiración a 60 minutos acotan la ventana de exposición si el correo es interceptado.
- Prioridad: Must
- Aceptación: enlace usado o expirado → mensaje de invalidez; tras el flujo, login inmediato con nueva contraseña.
- Verificación: Test unitario + prueba manual de flujo de correo
- Estado: verificado

REQ-F-005 (ex RF-05) — 2FA opcional basada en TOTP (RFC 6238), disponible solo para usuarios con identidad verificada.

- Rationale: Exigir 2FA solo a usuarios con identidad ya verificada evita añadir fricción a cuentas cuyo riesgo principal (fraude de identidad) el 2FA no mitiga; hacerlo opcional respeta la usabilidad del resto.
- Prioridad: Should

- Aceptación: código incorrecto/expirado → “Código inválido o expirado”; usuario no verificado no ve la opción.
- Verificación: Test (`TwoFactorServiceImplTest`, `TwoFactorController`)
- Estado: verificado

Módulo Perfiles, Verificación y Portafolio

REQ-F-006 (ex RF-06) — Verificación de mayoría de edad del Creador mediante documento de identidad analizado por servicio externo; el documento se elimina del almacenamiento tras la respuesta.

- Rationale: Operar con creadores menores de edad expone a la plataforma a riesgo legal y reputacional; delegar el análisis del documento a un servicio externo evita construir una capacidad propia de verificación de identidad, y eliminarlo tras la respuesta minimiza el dato personal retenido.
- Prioridad: Must
- Aceptación: aprobado → estado verificado en ≤ 60 s y notificación; minoría de edad → estado sin cambios y mensaje de rechazo; documento no accesible tras respuesta.
- Verificación: Test + inspección de almacenamiento
- Estado: verificado (flujo cubierto por `VerificacionServicioImplTest`, `VerificacionControladorTest` y `CertificadoIaRepositoryIT`; el proveedor de IA se sustituye por un doble en las pruebas, de modo que la integración con el servicio real queda como riesgo declarado en §6)

REQ-F-007 (ex RF-07) — Verificación de certificados profesionales por IA; puntaje $\geq$ umbral configurable (0.75 por defecto) habilita sello de verificación.

- Rationale: Sin verificación de certificados, cualquier creador podría reclamar credenciales profesionales no comprobadas, degradando la confianza del catálogo; un umbral configurable permite ajustar la exigencia sin desplegar código nuevo.
- Prioridad: Should
- Aceptación: puntaje $\geq$ umbral → sello inmediato; puntaje menor → sin sello + notificación; umbral configurable sin cambio de código.
- Verificación: Test + demostración
- Estado: verificado

REQ-F-008 (ex RF-08) — Personalización de perfil público: foto (JPG/PNG ≤ 5 MB), biografía (≤ 500 caracteres, sin teléfono/correo), hasta 3 URLs de redes sociales.

- Rationale: El perfil público es la vitrina de venta del Creador; limitar biografía e imagen y prohibir contacto directo evita que el perfil se use para negociar y cobrar fuera de la plataforma, eludiendo la comisión de REQ-F-021.
- Prioridad: Must
- Aceptación: biografía con contacto directo → rechazo; imagen > 5 MB → rechazo; URL válida se muestra como enlace.
- Verificación: Test (`CreatorProfileController`)
- Estado: verificado

REQ-F-009 (ex RF-09) — El perfil público muestra seguidores, servicios activos, calificación promedio y estado de verificación; cualquier usuario autenticado puede seguir/dejar de seguir.

- Rationale: Mostrar métricas de reputación (seguidores, calificación, verificación) directamente en el perfil es lo que permite a un Cliente decidir con quién contratar sin salir de la plataforma.
- Prioridad: Must
- Aceptación: contador se actualiza de inmediato al seguir/dejar de seguir; usuario no autenticado es redirigido a login.
- Verificación: Test + demostración
- Estado: verificado

REQ-F-010 (ex RF-10) — Comentarios en ítems de portafolio; el Creador puede eliminarlos (borrado lógico, no visibles en vista pública, consultables por el administrador).

- Rationale: los comentarios son la única forma de interacción social directa sobre una obra concreta (a diferencia de los seguidores, que son sobre el perfil); el borrado lógico permite auditar abuso sin destruir evidencia.
- Prioridad: Should
- Aceptación: un comentario eliminado por su autor o por el dueño del portafolio deja de aparecer en la vista pública pero sigue siendo consultable por el Administrador vía GET /api/v1/admin/comentarios; un usuario no autenticado no puede comentar.
- Verificación: Test (ComentarioPortafolioServiceImplTest, ComentarioPortafolioControladorTest, AdminComentarioControladorTest)
- Estado: verificado

Módulo Catálogo Dinámico de Servicios

REQ-F-011 (ex RF-11) — Publicación de ítems tipo Producto o Servicio, con precio (≥ 0.01 USD), al menos una imagen (≤ 10 MB) y descripción (20–2000 caracteres) obligatorios.

- Rationale: un precio de 0 o negativo rompe el cálculo de comisión (REQ-F-021) y el flujo de pago (REQ-F-020); una descripción mínima evita publicaciones vacías que degradan la calidad del catálogo público.
- Prioridad: Must
- Aceptación: precio menor a 0.01 USD → rechazo (“El precio debe ser de al menos 0.01 USD”); un ítem sin al menos una subcategoría asociada → rechazo (“El servicio necesita al menos una subcategoría”).
- Verificación: Test (ServicioCatalogoServicioImplTest)
- Estado: verificado

REQ-F-012 (ex RF-12) — Hasta 10 atributos personalizados por ítem; formularios adaptados dinámicamente a la categoría del Creador.

- Rationale: cada categoría de servicio (ilustración, desarrollo, música) necesita capturar datos distintos; un límite fijo evita que el formulario dinámico crezca sin control y degrade la experiencia de publicación.
- Prioridad: Must
- Aceptación: agregar un atributo número 11 al mismo ítem → rechazo (“Se ha alcanzado el límite de 10 atributos personalizados por ítem”); agregar un atributo ya asociado al mismo ítem → rechazo (“ya se encuentra asociado a este servicio”).
- Verificación: Test (ServicioCatalogoServicioImplTest)
- Estado: verificado

REQ-F-013 (ex RF-13) — Motor de búsqueda con filtros por categoría, subcategoría, rango de precio y etiquetas, más búsqueda textual sobre título/descripción; edición de ítems en cualquier momento.

- Rationale: con un catálogo de tamaño creciente, filtrar solo por categoría no basta para que un Cliente encuentre un servicio concreto; combinar todos los filtros en una sola consulta (Specification API) evita N llamadas sucesivas del frontend.
- Prioridad: Must
- Aceptación: los filtros de categoría, subcategoría, rango de precio, etiquetas y texto son combinables entre sí (AND); la búsqueda textual es insensible a mayúsculas y busca coincidencia parcial en título o descripción; sin ningún filtro, devuelve el catálogo completo en estado ACTIVO.
- Verificación: Test (Specification API — specification/catalogo)
- Estado: verificado

Módulo Comunicación y Notificaciones

REQ-F-014 (ex RF-14) — Mensajería interna en tiempo real vía WebSocket; sala de chat creada automáticamente al firmar el contrato; se cierra cuando el Cliente aprueba el entregable.

- Rationale: centralizar la comunicación dentro de la plataforma (en vez de intercambiar contactos externos) es la base del modelo de negocio; permite moderar contenido (REQ-F-015) y mantiene la evidencia

del acuerdo dentro del sistema.

- Prioridad: Must
- Aceptación: enviar un mensaje en una sala cerrada → rechazo (“Esta sala ha sido cerrada”); un usuario sin acceso al pedido no puede leer ni escribir en su chat (“No tiene acceso al chat de este pedido”).
- Verificación: Test + prueba de carga WebSocket (ver REQ-NF-005)
- Estado: verificado

REQ-F-015 (ex RF-15) — Análisis de contenido de mensajes para detectar teléfonos/correos; bloqueo de entrega y aviso; suspensión de 15 días tras 3 infracciones en 30 días. La consulta administrativa de infracciones y la reversión manual de una suspensión se especifican en REQ-F-033.

- Rationale: permitir el intercambio de contacto directo en el chat rompe el modelo de comisión (las partes podrían negociar fuera de la plataforma); la ventana de 30 días evita que una infracción antigua penalice indefinidamente a un usuario que ya corrigió su comportamiento.
- Prioridad: Must
- Aceptación: un mensaje con teléfono o correo detectado se bloquea antes de persistirse y genera una infracción; la tercera infracción dentro de una ventana de 30 días suspende la cuenta 15 días.
- Verificación: Test
- Estado: verificado

REQ-F-016 (ex RF-16) — Cuestionario (briefing) configurable (hasta 10 preguntas) asociado a un servicio del Creador; si el servicio tiene uno asignado, el Cliente lo responde al crear el pedido (obligatorio antes de que el pedido se registre); respuestas no editables tras el envío. Un servicio sin cuestionario asignado no bloquea la creación del pedido.

- Rationale: (ver también `CHANGELOG-REQ.md v1.1.0`) ligar el cuestionario al servicio y exigirlo en la creación del pedido garantiza que el Creador reciba el contexto que pidió, en vez de depender de que lo solicite manualmente después.
- Prioridad: Must
- Aceptación: crear un pedido de un servicio con cuestionario asignado sin respuestas → rechazo (“Este servicio tiene un cuestionario: responde todas sus preguntas para crear el pedido”); una plantilla de cuestionario no puede superar 10 preguntas (“Una plantilla no puede tener más de 10 preguntas”); la validación ocurre antes de persistir el pedido, para no dejarlo a medias.
- Verificación: Test
- Estado: verificado

Módulo Legal, Entregables y Finanzas

REQ-F-017 (ex RF-17) — Generación automática de contrato HTML desde la plantilla asignada al servicio del pedido (catálogo de plantillas curado por Administrador, gestionable con alta, edición y desactivación), o la plantilla predeterminada si el servicio no tiene una propia, sustituyendo variables (partes, servicio, precio, revisiones, fecha).

- Rationale: (ver también `CHANGELOG-REQ.md v1.1.0`) un catálogo curado por Administrador permite personalizar el texto legal por tipo de servicio sin exponer a la plataforma a cláusulas no revisadas escritas por cualquier Creador; debe existir siempre una plantilla predeterminada para que ningún servicio quede sin contrato generable.
- Prioridad: Must
- Aceptación: crear una plantilla con una versión legal ya existente → rechazo (“Ya existe una plantilla con la version legal...”); intentar desmarcar la única plantilla predeterminada sin marcar otra antes → rechazo; desactivar la plantilla predeterminada → rechazo (“No se puede desactivar la plantilla predeterminada; marca otra como predeterminada primero”).
- Verificación: Test (`ContractController`, `ContractTemplateAdminServiceImplTest`)
- Estado: verificado

REQ-F-018 (ex RF-18) — Firma electrónica como acción explícita de cada parte; el pedido no avanza sin ambas firmas; PDF descargable con hashes de firma.

- Rationale: exigir una acción explícita de firma (no un checkbox implícito al avanzar de etapa) deja evidencia inequívoca de que cada parte leyó y aceptó el contrato antes de que haya dinero en juego.
- Prioridad: Must
- Aceptación: crear un segundo contrato para un pedido que ya tiene uno → rechazo (“Ya existe un contrato para este pedido”); una parte que intenta firmar dos veces → rechazo (“El creador ya firmo este contrato” / “El cliente ya firmo este contrato”).
- Verificación: Test
- Estado: verificado

REQ-F-019 (ex RF-19) — Flujo de trabajo del pedido por etapas configurables según categoría; cada transición registrada con marca de tiempo; vista de seguimiento en tiempo real para el Cliente.

- Rationale: cada categoría de servicio tiene un ciclo de trabajo distinto (un diseño gráfico no pasa por las mismas etapas que un desarrollo de software); permitir que el flujo se configure por categoría, en vez de ser único y fijo, refleja esa diferencia sin necesitar código nuevo por categoría.
- Prioridad: Must
- Aceptación: un pedido de un flujo sin etapas configuradas no puede avanzar (“no tiene etapas configuradas”); dos flujos no pueden compartir nombre (“Ya existe un flujo de trabajo con el nombre...”); una etapa no puede intercambiarse consigo misma ni con una etapa de otro flujo.
- Verificación: Test (`FlujoTrabajoServicioImplTest`, `PedidoServicioImplFlujoTest`)
- Estado: verificado

REQ-F-020 (ex RF-20) — Generación de enlace de pago vía PayPal Orders v2 al iniciar pedido; actualización de estado de fondos al recibir webhook confirmado.

- Rationale: Sin un enlace de pago generado automáticamente al iniciar el pedido, el cobro dependería de un proceso manual fuera de la plataforma, rompiendo la trazabilidad exigida por el patrón escrow.
- Prioridad: Must
- Aceptación: al iniciar un pedido se genera una orden de PayPal y un enlace de pago asociado; al recibir un webhook confirmado con firma válida, el estado de los fondos del pago en garantía cambia de **Pendiente** a **Retenido**.
- Verificación: Test (`PayPalWebhookController`, sandbox)
- Estado: verificado (el webhook y su validación de firma están cubiertos por `PagoServicioImplWebhookTest`; queda como trabajo futuro la validación end-to-end contra el sandbox real de PayPal)

REQ-F-021 (ex RF-21) — Entrega con marca de agua para previsualización; aprobación del Cliente libera fondos y habilita descarga limpia; comisión de plataforma registrada automáticamente.

- Rationale: La marca de agua permite al Cliente evaluar el entregable antes de pagar sin poder usarlo todavía; ligar la liberación de fondos a su aprobación es lo que materializa el patrón escrow declarado en el alcance (§1.2).
- Prioridad: Must
- Aceptación: la comisión de plataforma es 10% del monto bruto (configurable vía `PLATAFORMA_COMISION_TASA`), registrada automáticamente en la transacción al aprobar el entregable.
- Verificación: Test (`DeliverableController`, `PaymentController`)
- Estado: verificado

REQ-F-022a (ex RF-22) — Cargo configurable por revisión adicional de un pedido que supera las revisiones incluidas.

- Rationale: sin un cargo por revisión adicional, el Creador no tiene forma de monetizar el tiempo de trabajo extra que pide un Cliente más allá de lo acordado originalmente en el pedido.
- Prioridad: Should
- Aceptación: el monto del cargo es configurable sin cambio de código; el ticket de revisión queda asociado al pedido correspondiente.
- Verificación: Test (`TicketRevisionServicioImplTest`)
- Estado: verificado

REQ-F-022b (ex RF-22) — Un ticket de revisión que supera el límite de revisiones configurado genera un nuevo enlace de pago para la revisión adicional.

- Rationale: cobrar la revisión adicional exige un medio de pago concreto; sin un enlace de pago generado automáticamente, el cargo configurado en REQ-F-022a no tiene forma de cobrarse.
- Prioridad: Should
- Aceptación: al superar el límite de revisiones configurado, el sistema genera un nuevo enlace de pago asociado al ticket de revisión.
- Verificación: Test (`RevisionTicketPaymentServiceImplTest`, `RevisionTicketServiceImplTest#crearTicketRevisi`)
- Estado: implementado (`RevisionTicketPaymentServiceImpl` crea la orden PayPal automáticamente al crear el ticket cuando supera el límite, entidad propia `pagos_ticket_revision` para no competir con el escrow principal; falta una prueba contra el sandbox real de PayPal para subir a verificado — ver excepciones-estado.txt)

REQ-F-022c (ex RF-22) — Un ticket de revisión sin pago confirmado tras 48 horas se rechaza automáticamente.

- Rationale: sin un rechazo automático, un ticket de revisión impago quedaría indefinidamente abierto, bloqueando el avance del pedido sin que nadie lo resuelva.
- Prioridad: Should
- Aceptación: un ticket de revisión que no recibe confirmación de pago dentro de 48 horas desde su creación cambia automáticamente a un estado de rechazo.
- Verificación: Test (`RevisionTicketExpirationSchedulerTest`, `RevisionTicketExpirationServiceTest`, `TicketRevisionRepositoryIT`)
- Estado: implementado (`RevisionTicketExpirationScheduler` nuevo, umbral configurable vía `ticketrevision.expiracion-horas`, default 48h; solo afecta tickets que sí generaron cargo y siguen sin pago confirmado — un ticket que nunca superó el límite no se auto-rechaza; falta una prueba contra el sandbox real de PayPal para subir a verificado — ver excepciones-estado.txt)

Módulo Social, Comunidad y Sorteos

REQ-F-023 (ex RF-23) — Creación de sorteos (título, premio, ganadores, fechas, requisito de seguidor); selección aleatoria automática de ganadores al cierre.

- Rationale: un sorteo es una herramienta de crecimiento orgánico de la base de seguidores del Creador; automatizar la selección de ganadores al cierre evita disputas sobre la aleatoriedad del proceso.
- Prioridad: Could
- Aceptación: inscribirse en un sorteo que no está activo, que aún no comenzó, o cuyo periodo de inscripción ya finalizó → rechazo en cada caso; inscribirse dos veces en el mismo sorteo → rechazo (“Ya estás inscrito en este sorteo”); cancelar la inscripción en un sorteo ya finalizado → rechazo.
- Verificación: Test + demostración
- Estado: verificado

3.1 Requisitos adicionales (post v1.0.0)

Los 23 requisitos anteriores (REQ-F-001 a REQ-F-023) son el corpus original heredado de la Entrega 1A. Los ocho requisitos siguientes (REQ-F-024 a REQ-F-031) se incorporan en v1.1.2 tras una auditoría del SRS contra el código real: documentan módulos que ya estaban implementados y probados en el backend, pero que no tenían ningún requisito funcional que los especificara. No representan alcance nuevo del sistema, sino especificación que le faltaba a alcance ya construido — ver `CHANGELOG-REQ.md` v1.2.0 para el detalle de por qué cada uno faltaba.

Módulo Financiero — Retiros y Auditoría de Pagos

REQ-F-024 — El Creador debe poder configurar su correo de PayPal y solicitar el retiro de su saldo disponible; el Auditor Financiero (o Administrador) revisa la cola de solicitudes y las aprueba, rechaza o reintenta.

- Rationale: el escrow de REQ-F-021 retiene fondos hasta la aprobación del entregable, pero sin un flujo de retiro el Creador nunca recibe el dinero fuera de la plataforma; es el cierre natural del ciclo de pago.
- Prioridad: Must
- Aceptación: solicitar retiro sin correo de PayPal configurado → rechazo; monto menor al mínimo configurado (USD 10.00 por defecto, `RETIROS_MONTO_MINIMO`) → rechazo; monto mayor al saldo disponible → rechazo; ya existe una solicitud en curso → rechazo; aprobar ejecuta el pago vía PayPal Payouts y cambia el estado; rechazar exige nota administrativa; reintentar solo aplica a solicitudes en estado “Fallido”.
- Verificación: Test (`SolicitudRetiroServicioImplTest`, `SolicitudRetiroServicioImplPayoutTest`, `SolicitudRetiroControladorTest`, `SolicitudRetiroAdminControladorTest`, `SolicitudRetiroAutorizacionTest`, `SolicitudRetiroConcurrenciaIT`, `DatosPagoControladorTest`)
- Estado: verificado

REQ-F-025 — El Auditor Financiero (o Administrador) debe poder listar de forma paginada y filtrada los pagos en garantía (escrow), ver el detalle con su historial de transacciones, y obtener un resumen agregado de fondos por estado.

- Rationale: el patrón escrow retiene dinero de clientes; sin una pantalla de supervisión, nadie puede auditar cuánto dinero está retenido, por cuánto tiempo, ni en qué estado, lo cual es un riesgo financiero y de cumplimiento.
- Prioridad: Must
- Aceptación: acceso restringido a `PAGO_AUDITAR` o rol Administrador; el resumen agrega cantidad y monto total por estado; el detalle de un pago inexistente devuelve 404.
- Verificación: Test (`PagoGarantiaAuditoriaServicioImplTest`)
- Estado: verificado

Módulo de Reportes

REQ-F-026 — El Administrador (o titular de `TRANSACCION_VER/REPORTE_FINANCIERO_EXPORTAR/REPORTE_CONTRATO_EXPO`) debe poder consultar y exportar en CSV, XLSX o PDF: (a) el reporte de comisiones por creador (bruto, comisión, neto, detalle de transacciones) y (b) el listado de contratos formalizados (servicio, partes, precio, estado de firma).

- Rationale: estos reportes exponen datos financieros y de contratos de terceros; sin requisito propio, nadie declaró quién puede generarlos ni bajo qué límites.
- Prioridad: Should
- Aceptación: exportar con un número de transacciones que excede el tope de filas del formato solicitado → rechazo (`ExcepcionReglaNegocio`); el acceso a `/exportar` exige el permiso específico, distinto del permiso de solo lectura del reporte.
- Verificación: Test (`ReporteFinancieroServicioImplTest`, `ReporteFinancieroAutorizacionTest`, `ReporteContratoServicioImplTest`, `ReporteContratoAutorizacionTest`)
- Estado: verificado

Módulo de Notificaciones

REQ-F-027 — El sistema debe mantener un centro de notificaciones transversal por usuario, alimentado por eventos de otros módulos (pedidos, pagos, verificación, etc.); el usuario puede listar sus notificaciones paginadas, marcarlas como leídas (una o todas) y consultar el contador de no leídas.

- Rationale: varios módulos (verificación, pedidos, retiros) ya notifican al usuario en la práctica; sin este requisito, el mecanismo transversal que los sostiene no está especificado.
- Prioridad: Should
- Aceptación: listar devuelve solo las notificaciones del usuario autenticado; marcar como leída una notificación ajena → 404; el contador de no leídas baja a 0 tras “marcar todas”.
- Verificación: Test (`NotificacionServiceImplTest`, `NotificacionControladorTest`)
- Estado: verificado

Módulo Catálogo — Gestión Auxiliar

REQ-F-028 — El Creador debe poder crear categorías propias (quedan sin revisar hasta que un Moderador/Administrador las apruebe); el Moderador/Administrador debe poder gestionar categorías, subcategorías y etiquetas del catálogo (crear, editar, desactivar), revisar las pendientes de creadores, y moderar servicios ajenos quitándoles una subcategoría inapropiada.

- Rationale: REQ-F-011/012/013 dan por hecho un catálogo de categorías/subcategorías/etiquetas ya existente, pero ningún requisito especifica cómo se crean, revisan ni moderan esos catálogos, y hoy tienen CRUD y flujo de revisión propios.
- Prioridad: Must (es prerequisite operativo de REQ-F-011/012/013)
- Aceptación: una categoría creada por un Creador con `CATEGORIA_CREAR` queda pendiente de revisión y no aparece en el listado público hasta ser marcada revisada; quitar la última subcategoría de un servicio en moderación → rechazo (`ExcepcionReglaNegocio`).
- Verificación: Test (`CategoriaServicioImplTest`, `CategoriaControladorTest`, `CategoriaAutorizacionTest`, `EtiquetaServicioImplTest`, `ServicioCatalogoServicioImplTest` — cubre la moderación de subcategorías)
- Estado: verificado

Módulo Social — Reseñas

REQ-F-029 — El Cliente debe poder calificar (1-5 estrellas) y reseñar un pedido solo después de que el entregable fue liberado (aprobado); puede editar o eliminar su propia reseña; cualquier visitante puede consultar las reseñas y el promedio de un Creador.

- Rationale: REQ-F-009 ya menciona que el perfil “muestra... calificación promedio”, pero ese requisito solo cubre la exhibición, no el flujo completo de creación/edición/eliminación que alimenta ese promedio.
- Prioridad: Must
- Aceptación: crear reseña sobre un pedido sin entregable liberado → rechazo; crear una segunda reseña sobre el mismo pedido → rechazo (`ExcepcionRecursoDuplicado`); el promedio público se recalcula tras cada alta/edición/baja.
- Verificación: Test (`ResenaServiceImplTest`, `ResenaControladorTest`)
- Estado: verificado

Módulo Seguridad — Administración de Cuentas

REQ-F-030 — El Administrador (o titular de los permisos `USUARIO_*/ROL_GESTIONAR/SESION_REVOCAR`) debe poder listar, exportar, crear, editar, activar/desactivar/suspender, asignar roles, revocar sesiones y eliminar lógicamente cuentas de usuario desde el panel administrativo.

- Rationale: es distinto de REQ-F-002 (RBAC operativo por permiso): este requisito es la gestión del ciclo de vida de la cuenta en sí, con auto-protecciones (un administrador no puede desactivarse, cambiarse sus propios roles ni eliminarse a sí mismo).
- Prioridad: Should
- Aceptación: un administrador que intenta desactivar, cambiar roles o eliminar su propia cuenta → rechazo (`ExcepcionReglaNegocio`); revocar sesiones invalida inmediatamente cualquier JWT activo del usuario afectado.
- Verificación: Test (`AdminUserServiceImplTest`, `AdminControllerTest`)
- Estado: verificado

Módulo Auxiliar — Catálogo de Países y Portafolio Social

REQ-F-031 — El sistema debe mantener un catálogo maestro de países (CRUD administrativo, listado de activos) para uso en formularios de perfil/verificación, y permitir a cualquier usuario autenticado dar o quitar “me gusta” a un ítem de portafolio, con conteo público.

- Rationale: son dos capacidades menores, sin impacto financiero ni legal, agrupadas por bajo riesgo; documentarlas cierra la brecha de cobertura sin inflar la prioridad del corpus.
- Prioridad: Could
- Aceptación: dar like dos veces al mismo ítem → rechazo (`ExcepcionRecursoDuplicado`); el estado de like es consultable sin autenticación (usuario anónimo ve el conteo, no si “ya dio like”).
- Verificación: Test (`PaisServiceImplTest`, `PaisControllerTest`, `LikePortafolioServiceImplTest`, `LikePortafolioControladorTest`)
- Estado: verificado

Módulo Portafolio — Gestión de Obras (v1.3.0)

REQ-F-032 — El Creador debe poder subir, listar, actualizar y eliminar obras (ítems) de su portafolio, con un máximo de 50 obras por portafolio; la visibilidad de las obras respeta la del portafolio (público/privado) y el archivo se sirve siempre como descarga forzada, nunca renderizado inline.

- Rationale: un portafolio sin límite de obras degrada el rendimiento de la vista pública y facilita abuso de almacenamiento; servir un ítem con `Content-Disposition: attachment` evita que un SVG subido como obra se interprete como HTML en el dominio de la plataforma (XSS almacenado).
- Prioridad: Must
- Aceptación: subir una obra número 51 al mismo portafolio → rechazo; un usuario que no es dueño del portafolio no puede modificar ni eliminar sus obras; un portafolio no público solo es visible para su dueño; toda descarga de archivo lleva la cabecera `Content-Disposition: attachment`.
- Verificación: Test (`PortafolioItemControladorTest`, `PortafolioItemControladorRutasTest`, `PortafolioItemServicioImplTest`)
- Estado: verificado

Módulo Comunicación — Administración de Infracciones (v1.3.0)

REQ-F-033 — El Administrador (o titular de `INFRACCION_GESTIONAR`) debe poder listar todas las infracciones registradas en el sistema, consultar el historial de infracciones de un usuario específico, y revertir manualmente la suspensión de una cuenta.

- Rationale: REQ-F-015 detecta y suspende automáticamente, y ya cita `AdminViolationController` en `matriz.csv` como parte de su módulo, pero solo cubre (con prueba) el flujo de detección en el chat; los endpoints administrativos `listarInfracciones` y `revertirSuspension` no tienen historia, caso de uso ni prueba propios. Además, una suspensión automática puede ser un falso positivo (por ejemplo, un número de teléfono que en realidad forma parte del texto de un servicio); sin esta capacidad, revertirla exigiría acceso directo a la base de datos.
- Prioridad: Should
- Aceptación: el listado y el historial por usuario son de solo lectura; revertir la suspensión de una cuenta no suspendida no debe producir un estado inconsistente.
- Verificación: Test (`InfraccionServiceImplTest`, con los 3 endpoints cubiertos)
- Estado: verificado

4. Requisitos específicos — No funcionales

REQ-NF-001a (ex RNF-01) — Redirección forzada de HTTP a HTTPS (301).

- Rationale: sin redirección forzada, un cliente que escribe `http://` por error queda expuesto en texto plano durante ese primer contacto, antes de que el servidor tenga oportunidad de cifrar la conexión; forzar el salto a HTTPS en el primer byte cierra esa ventana de exposición.
- Prioridad: Must

- Aceptación: una solicitud HTTP a la raíz del dominio responde 301 con cabecera `Location` apuntando al equivalente HTTPS; ninguna ruta de la aplicación permanece accesible por HTTP sin esa redirección previa.
- Verificación: análisis externo (SSL Labs) + script re-ejecutable
- Estado: verificado (`scripts/verificar-tls-ssllabs.sh` registrado como `prueba_automatizada` — el script tiene asserts y exit code 0/1; análisis SSL Labs contra `artisync-frontend.onrender.com` ejecutado y archivado el 2026-09-10: `curl -i http://artisync-frontend.onrender.com` responde 301 a `https://`, grade A+ — ver `docs/mediciones/sec/ssl-labs/REPORTE-SSL-LABS.md`)

REQ-NF-001b (ex RNF-01) — Rechazo de conexiones con TLS inferior a 1.2.

- Rationale: TLS 1.0 y 1.1 tienen vulnerabilidades conocidas (POODLE, BEAST) y fueron retirados por los navegadores modernos; aceptarlos exponería a los usuarios a ataques de downgrade sobre una conexión que debería ser segura por diseño.
- Prioridad: Must
- Aceptación: un intento de conexión que fuerza TLS 1.0 o 1.1 es rechazado por el servidor; solo TLS 1.2 y TLS 1.3 son negociables.
- Verificación: análisis externo (SSL Labs) + script re-ejecutable
- Estado: verificado (`scripts/verificar-tls-ssllabs.sh` registrado como `prueba_automatizada`; SSL Labs solo reporta TLS 1.2 y TLS 1.3 como protocolos aceptados por el servidor, grade A+ — ver `docs/mediciones/sec/ssl-labs/REPORTE-SSL-LABS.md` y `ssllabs-api-response-20260910.json`)

REQ-NF-001c (ex RNF-01) — Preferencia de TLS 1.3 cuando el cliente lo soporta.

- Rationale: TLS 1.3 reduce la latencia del handshake y elimina por diseño los cifrados obsoletos que TLS 1.2 todavía permite negociar; preferirlo cuando el cliente lo soporta mejora seguridad y rendimiento sin ningún costo adicional.
- Prioridad: Must
- Aceptación: ante un cliente que ofrece TLS 1.3, el servidor lo selecciona en vez de negociar una versión anterior.
- Verificación: análisis externo (SSL Labs) + script re-ejecutable
- Estado: verificado (`scripts/verificar-tls-ssllabs.sh` registrado como `prueba_automatizada`; el servidor negocia TLS 1.3 cuando el cliente lo ofrece — `openssl s_client` sin forzar versión negocia TLSv1.3/TLS_AES_256_GCM_SHA384, `forwardSecrecy=4` en SSL Labs — ver `docs/mediciones/sec/ssl-labs/REPORTE-SSL-LABS.md`)

REQ-NF-002 (ex RNF-02) — Contraseñas con hash bcrypt, factor de coste ≥ 10 ; nunca texto plano.

- Rationale: un factor de coste bajo permite crackear por fuerza bruta un volcado de la tabla de usuarios en tiempo razonable; bcrypt con coste ≥ 10 encarece ese ataque exponencialmente sin degradar de forma perceptible el login legítimo.
- Prioridad: Must
- Aceptación: toda contraseña almacenada en `usuarios.contrasena_hash` es un hash bcrypt con factor de coste ≥ 10 ; ninguna ruta del sistema persiste ni registra la contraseña en texto plano, logs incluidos.
- Verificación: inspección de BD
- Estado: verificado

REQ-NF-003 (ex RNF-03) — JWT firmado HS256 con clave ≥ 256 bits en variable de entorno (nunca en código/repositorio); rechazo de firma inválida con 401.

- Rationale: una clave de firma corta o embebida en el repositorio es trivialmente comprometible; separarla en variable de entorno y exigir 256 bits mínimos hace inviable falsificar tokens por fuerza bruta o por lectura del código fuente.
- Prioridad: Must
- Aceptación: un token con firma alterada o firmado con una clave distinta responde 401 antes de procesar la solicitud; la clave de firma no aparece en ningún archivo versionado del repositorio.
- Verificación: análisis + test
- Estado: verificado (clave vía `.env`, validada al arrancar; los claims `iss`, `aud`, `nbf` y `jti` se emiten y se validan al parsear — ver OBS-AUTO-01 y OBS-AUTO-08 en `docs/observaciones/OBSERVACIONES.md`)

REQ-NF-004 (ex RNF-04) — LCP del catálogo $\leq 2s$ bajo 4G simulada con ≥ 20 servicios publicados.

- Rationale: el catálogo es la puerta de entrada del producto para un Cliente que aún no decidió contratar; un tiempo de carga inicial lento eleva la tasa de abandono antes de que vea ningún servicio publicado.
- Prioridad: Should
- Aceptación: medido con Lighthouse bajo perfil de red 4G simulada y con al menos 20 servicios publicados en el catálogo, el LCP es $\leq 2s$.
- Verificación: Lighthouse
- Estado: implementado (LCP medido $\sim 2.8s$)

REQ-NF-005 (ex RNF-05) — WebSocket: ≥ 10 conexiones simultáneas sin degradación; latencia extremo-a-extremo $\leq 500ms$ en red local.

- Rationale: REQ-F-014 exige centralizar toda la comunicación del pedido en el chat interno; si el canal no soporta múltiples conexiones simultáneas sin degradar la latencia, ese requisito de negocio se vuelve inviable a partir de cierto volumen de usuarios concurrentes.
- Prioridad: Should
- Aceptación: con ≥ 10 conexiones STOMP simultáneas en red local, la latencia extremo-a-extremo del envío de un mensaje no supera 500ms.
- Verificación: ChatWebSocketLoadIT (prueba automatizada, 10 conexiones STOMP reales $\times$ 5 rondas)
- Estado: verificado (p95=346ms, máximo=346ms, 0 conexiones fallidas — ver docs/mediciones/ws/REPORTE-WS.md. La prueba expuso y forzó a corregir un defecto real: @AuthenticationPrincipal no se resolvía en @MessageMapping sin AuthenticationPrincipalArgumentResolver + SecurityContextHolderInterceptor en WebSocketConfig, dejando roto el envío de chat por WebSocket para cualquier cliente real)

REQ-NF-006 (ex RNF-06) — Generación de contrato PDF $\leq 5s$ bajo carga normal.

- Rationale: el pedido no puede avanzar sin que ambas partes firmen el contrato (REQ-F-018); una generación lenta del PDF bloquea todo el flujo de contratación justo en el momento en que el Cliente ya decidió comprar.
- Prioridad: Should
- Aceptación: bajo carga normal, la generación del PDF del contrato completa en $\leq 5s$.
- Verificación: ContratoPdfTimingIT (prueba automatizada, 5 corridas contra Postgres real)
- Estado: verificado (1218/29/30/20/20 ms, media 263.4ms — ver docs/mediciones/perf/REPORTE-PDF-CONTRATO.md)

REQ-NF-007 (ex RNF-07) — Interfaz sin desbordamiento horizontal en 360/768/1440px; controles operables táctilmente ($\geq 44px$).

- Rationale: sin una interfaz operable en pantallas pequeñas, una parte significativa de los Clientes que navegan desde el móvil no puede completar el flujo de contratación, afectando directamente el modelo de negocio de doble lado.
- Prioridad: Should
- Aceptación: en 360px, 768px y 1440px de ancho, ninguna vista principal produce scroll horizontal, y todo control interactivo mide al menos $44 \times 44px$.
- Verificación: DevTools — sin scroll horizontal visible en 360/768/1440px; todo control interactivo mide $\geq 44 \times 44px$ en el inspector
- Estado: implementado

REQ-NF-008 (ex RNF-08) — Formularios de catálogo dinámicos sin recarga; flujo de contratación en ≤ 5 pantallas.

- Rationale: un flujo de contratación con demasiados pasos o con recargas de página completas eleva la fricción justo antes de que el Cliente pague, aumentando el riesgo de abandono en el momento de mayor valor del negocio.
- Prioridad: Should
- Aceptación: completar el flujo de contratación de principio a fin no requiere recargar la página en ningún punto, y visita como máximo 5 pantallas distintas.
- Verificación: prueba manual — completar el flujo de contratación de principio a fin sin recarga de página, contando el número de pantallas distintas visitadas

- Estado: implementado

REQ-NF-009 (ex RNF-09) — Disponibilidad durante semanas de evaluación 16-17; reinicio automático ante fallos.

- Rationale: la Entrega Final se evalúa en vivo durante las semanas 16-17; una caída de contenedor que no se recupera sola durante la evaluación puede costar la calificación de funcionalidades que en realidad sí funcionan, por una interrupción de infraestructura ajena al código.
- Prioridad: Must
- Aceptación: los servicios declaran una política de reinicio automático y healthcheck; ante la caída de un contenedor, el servicio se recupera sin intervención manual dentro de una ventana razonable.
- Verificación: demostración (ps aux, healthcheck Docker)
- Estado: implementado (los cinco servicios de `artisync/docker-compose.yml` declaran `restart: unless-stopped` y healthcheck; demostración real ejecutada el 2026-09-10 contra el entorno local — `docker kill` sobre `pfc_backend` y `pfc_postgres` **no** disparó el reinicio automático en ninguno de los dos casos, ver `docs/mediciones/resiliencia/REPORTE-RECUPERACION.md`. Hallazgo activo, no evidencia pendiente; no aplica al despliegue real en Render, que no usa esta política de `docker-compose` — excepción declarada en `docs/trazabilidad/excepciones-estado.txt`)

REQ-NF-010 (ex RNF-10) — Módulos WebSocket, REST y generación de PDF desacoplados (sin imports cruzados directos).

- Rationale: acoplar los módulos de WebSocket, REST y generación de PDF dificultaría probar, desplegar o reemplazar uno sin arrastrar cambios en los otros dos, elevando el costo de mantenimiento a medida que el sistema crece.
- Prioridad: Should
- Aceptación: la clase de generación de PDF no importa directamente clases de los paquetes de WebSocket ni de ningún controlador REST.
- Verificación: inspección de dependencias — `grep` de imports en la clase `legal.impl.PdfGenerationServiceImpl`: solo importa su propia interfaz y la librería de generación de PDF, sin ningún import de `service.comunicacion` (WebSocket) ni de un controlador REST; el desacople es de esa clase específica, no de todo el paquete `legal` (otras clases del mismo paquete, como `PaymentServiceImpl`, sí dependen de `comunicacion` para notificaciones)
- Estado: implementado (paquetes separados por módulo)

REQ-NF-011 (ex RNF-11) — Archivos binarios en almacenamiento externo compatible con S3; sin archivos locales en el servidor.

- Rationale: guardar archivos en el disco del servidor de aplicación los pierde en cualquier redeploy o reinicio de contenedor, dado que el filesystem del hosting usado no es persistente, y además impide escalar horizontalmente sin sincronizar discos entre instancias.
- Prioridad: Must
- Aceptación: la variable de entorno de proveedor de almacenamiento resuelve a un proveedor externo compatible con S3 en el entorno auditado, y la URL de descarga de un documento recién subido apunta a ese proveedor, no a una ruta local del servidor.
- Verificación: inspección de configuración y de URL servida — confirmar que `documentos.proveedor` (`DOCUMENTOS_PROVEEDOR`) resuelve a `azure` en el entorno auditado, y que la URL de descarga de un documento recién subido apunta a un dominio `*.blob.core.windows.net` (Azure), no a una ruta local del servidor (`/uploads/...` o equivalente)
- Estado: verificado (se fijó el valor real de `AZURE_STORAGE_CONNECTION_STRING` en el dashboard de Render, se redespiegó y se confirmó la carga y descarga con una URL real servida desde Azure Blob Storage, cerrando el incumplimiento activo previo — evidencia archivada en `docs/despliegue/EVIDENCIA-AZURE.md`)

REQ-NF-012 (ex RNF-12) — Bloqueo de registro a menores de 18; checkbox obligatorio de términos y privacidad.

- Rationale: operar con menores de edad sin controles de registro expone a la plataforma a responsabilidad

legal, dado el alcance geográfico inicial declarado en §2.4 (Ecuador, validación de mayoría de edad).

- Prioridad: Must
- Aceptación: un registro con fecha de nacimiento que resulte en menos de 18 años es rechazado; el registro no se completa sin marcar el checkbox de términos y privacidad.
- Verificación: test + inspección HTML
- Estado: verificado

REQ-NF-013 (ex RNF-13) — Auditoría inmutable de transiciones de pedido y transacciones; exportación CSV por el administrador.

- Rationale: sin una bitácora que ni siquiera un administrador pueda alterar o borrar, cualquier disputa sobre quién hizo qué y cuándo —crítico en un sistema con dinero retenido en escrow— dependería de la palabra de quien tuviera acceso a la base de datos, incluido un atacante interno.
- Prioridad: Must
- Aceptación: un intento de UPDATE, DELETE o TRUNCATE sobre la tabla de eventos de auditoría, incluso con la cuenta de aplicación, es rechazado por el motor de base de datos.
- Verificación: test (UPDATE/DELETE/TRUNCATE → error de base de datos)
- Estado: verificado (además de `historial_estados_pedido` (dominio, insert-only — PATCH/DELETE responden 403), existe desde V15__modulo_auditoria.sql una bitácora transversal `auditoria_eventos` con trigger PL/pgSQL que bloquea UPDATE/DELETE/TRUNCATE (SQLState 42501) y GRANT restringido a SELECT, INSERT para la cuenta de aplicación, alimentada por un aspecto AOP (@Auditable) sobre 7 módulos — incluido FINANZAS (PAGO_ORDEN_CREAR, PAGO_WEBHOOK_RECIBIR, PAGO_CANCELAR, PAGO_RECONCILIAR, RETIRO_DATOS_PAGO_ACTUALIZAR), que es lo que cubre hoy “transacciones” en el enunciado —, expuesta en `/api/v1/admin/auditoria` con listado filtrado, detalle y exportación CSV/XLSX/PDF vía `/api/v1/admin/auditoria/exportar`; verificado con `EventoAuditoriaInmutabilidadIT` contra PostgreSQL real. **Corrección (ronda de revisión externa, 2026-09-11):** el enunciado citaba `AuditControlador` como “el exportador de transacciones” y un endpoint `GET /api/v1/admin/transacciones/{idPerfil}/csv`; esa clase (paquete `controller.social`, junto con `AuditService/AuditServiceImpl` del mismo paquete) fue eliminada el 2026-08-26 (commit 6bca09b8, “Elimina el modulo legacy de auditoria social... reemplazado por el modulo de auditoria actual”) y el endpoint nunca existió bajo esa ruta — la cita quedó huérfana en `SRS.md/matriz.csv` durante más de dos semanas sin que `validate-traceability.sh` lo detectara, porque el script nunca compara contra el código real. Se retiró la cita muerta; la cobertura de “transacciones” que el enunciado promete la sostiene la bitácora transversal filtrable por módulo FINANZAS, no una clase dedicada)

REQ-NF-014 (ex RNF-14) — Integración exclusiva con PayPal Orders v2; credenciales en variables de entorno; verificación de firma de webhook.

- Rationale: integrar más de un proveedor de pagos multiplicaría la superficie de riesgo financiero y de cumplimiento sin necesidad real en un proyecto de alcance acotado; exigir credenciales fuera del código y verificar la firma del webhook evita que un tercero falsifique notificaciones de pago.
- Prioridad: Must
- Aceptación: un webhook con firma inválida o ausente es rechazado antes de procesar cualquier cambio de estado de fondos; las credenciales de PayPal no aparecen en ningún archivo versionado.
- Verificación: inspección de Git + simulación de webhook inválido
- Estado: verificado (credenciales de PayPal vía `.env`; `PagoServicioImplWebhookTest` cubre firma inválida, cabeceras ausentes, payload ilegible y ausencia de `webhook-id`, además del camino feliz)

4.1 Requisitos no funcionales adicionales (post v1.0.0)

Igual que en §3.1, los tres requisitos siguientes (REQ-NF-015 a REQ-NF-017) se incorporan en v1.1.2 tras auditar el SRS contra la evidencia ya archivada en `docs/mediciones/` y el código real: formalizan umbrales y comportamientos que el equipo ya perseguía o medía de facto, pero que ningún requisito capturaba.

REQ-NF-015 — Ante indisponibilidad de Redis, los servicios que dependen de él (cuota de intentos de login, lista de revocación de JWT, caché del catálogo) deben degradar de forma explícita y documentada (fail-open

o fail-closed según el servicio, ver ADR-004), nunca fallar en silencio; el TTL de la caché del catálogo debe ser configurable por variable de entorno.

- Rationale: si el catálogo, la cuota de login o la revocación de JWT dependen de Redis y este cae sin un comportamiento definido, el sistema podría fallar en silencio (ej. dejar pasar intentos de login ilimitados) o caerse por completo por una dependencia que debería ser un acelerador, no un punto único de fallo.
- Prioridad: Must
- Aceptación: ante la caída simulada de Redis, cada servicio dependiente degrada según el comportamiento documentado en su ADR (fail-open o fail-closed), nunca lanza un error no controlado; el TTL de la caché del catálogo cambia con la variable de entorno sin requerir cambio de código.
- Verificación: Test (simulación de caída de Redis)
- Estado: verificado (`IntentosAutenticacionServiceTest` prueba explícitamente los dos escenarios “Redis caído, fail-open” para verificación de cuota y limpieza; `app.cache.catalogo.ttl-seconds` es configurable vía `CATALOGO_CACHE_TTL`, ver `docs/adr/adr-004-estrategia-cache.md`)

REQ-NF-016 — La cobertura de código del backend (líneas) debe ser $\geq 70\%$ según JaCoCo, medida sobre la rama principal antes de cada entrega.

- Rationale: un backend sin piso de cobertura medido puede acumular código sin ninguna prueba que lo respalde, especialmente en capas que crecen rápido (como ocurrió con V45-V48); fijar un umbral verificable en cada entrega es lo que convierte “tenemos pruebas” en una afirmación auditable.
- Prioridad: Should
- Aceptación: el reporte JaCoCo de la rama principal, medido antes de la entrega, muestra cobertura de líneas $\geq 70\%$ a nivel global del backend.
- Verificación: `docs/mediciones/jacoco/REPORTE-JACOCo.md`
- Estado: verificado (82,93% líneas / 71,50% ramas, medición del 2026-09-11 ronda 2; el requisito solo exige líneas $\geq 70\%$, y se cumple con margen. Nota: una ronda anterior el mismo día había detectado la cobertura de ramas de la capa de Controladores en 67,07% tras incorporar las funcionalidades V45-V48 sin pruebas equivalentes — brecha declarada para OBS-P1-01 — pero se cerró el mismo día (86,18% líneas / 78,05% ramas en Controladores tras añadir pruebas para los 5 controladores y `BackupServiceImpl` sin cobertura propia); las tres capas superan hoy el 70% en líneas y ramas)

REQ-NF-017 — La usabilidad percibida del frontend, medida con System Usability Scale (SUS) sobre una muestra representativa de usuarios, debe alcanzar un puntaje $\geq 68/100$.

- Rationale: una plataforma que conecta clientes y creadores depende de que ambos puedan usarla sin fricción; sin un umbral de usabilidad medido con un instrumento estándar, una interfaz confusa podría pasar inadvertida hasta que ya esté afectando la adopción real.
- Prioridad: Should
- Aceptación: el puntaje SUS agregado sobre una muestra representativa de usuarios es $\geq 68/100$.
- Verificación: `docs/mediciones/sus/REPORTE-SUS.md`
- Estado: implementado (no cumple el umbral: 61,25/100 medido el 2026-08-16, calificación “D”; por debajo de 68 — brecha reconocida, excepción declarada en `docs/trazabilidad/excepciones-estado.txt`)

4.2 Requisitos no funcionales adicionales (v1.3.0)

Los ocho requisitos siguientes (REQ-NF-018 a REQ-NF-025) se incorporan en v1.3.0 tras una segunda auditoría externa del SRS contra el código real: cubren protección de datos personales, robustez de pagos, retención documental, contraseñas, límite de tasa, accesibilidad, respaldo de base de datos y auto-revocación de sesiones — capacidades con implicación legal, financiera o de seguridad que ningún requisito anterior capturaba. Ver `CHANGELOG-REQ.md` v1.3.0 para el detalle de por qué cada uno faltaba.

REQ-NF-018 — El sistema debe extender `docs/basedatos/POLITICA-RETENCION.md` para declarar y cumplir un período de retención sobre las categorías de datos personales sensibles que hoy quedan fuera de ella (documentos de identidad, certificados profesionales, datos de pago del creador, contratos firmados, mensaje-ría privada), debe ofrecer al titular un mecanismo real de supresión de sus datos a solicitud (distinto de la baja lógica de cuenta que ya existe), y debe mantener minimizados los datos personales enviados a servicios

externos: el servicio de verificación de identidad (REQ-F-006) y de certificados (REQ-F-007), que hoy ya solo reciben prompt + imagen.

- Rationale: la política de retención actual (`docs/basedatos/POLITICA-RETENCION.md`) cubre datos técnicos de sesión (`sesiones_usuario`, `tokens_recuperacion`, `codigos_respaldo_2fa`, `notificaciones_sistema`, 90 días) pero excluye por diseño los datos personales de mayor sensibilidad del corpus; `docs/etica/ETHICS.md` ya reconoce la tensión entre auditoría inmutable y derecho al olvido sin resolverla en código. Es una obligación legal en el marco de protección de datos personales aplicable (Ecuador: Ley Orgánica de Protección de Datos Personales), no solo buena práctica.
- Prioridad: Must
- Aceptación: `POLITICA-RETENCION.md` declara un período de retención verificable para documentos de identidad, certificados, datos de pago, contratos y mensajería; un usuario puede solicitar la supresión de sus datos personales (más allá de desactivar la cuenta) y el sistema la ejecuta o declara la excepción legal que la impide (p. ej. registros contables); se mantiene verificado que el payload enviado a los servicios externos de IA no incorpora campos identificativos adicionales a prompt + imagen.
- Verificación: inspección de código (payload minimizado, verificado en `GeminiIaService/NvidiaIaService`) + `docs/basedatos/POLITICA-RETENCION.md` extendida a `certificados_ia`, `datos_pago_creador` y `contratos` + `PrivacidadServiceImplTest` (16 casos unitarios con Mockito) + `PrivacidadServiceImplIT` (prueba de integración contra PostgreSQL real: bloqueo pesimista de `findByIdParaAnonimizar`, detección de pedido en curso vía `existsByFlujoIdFlujoAndEtapaIdEtapaAndEsEtapaFinalTrue`, y excepción legal por fondos retenidos, las tres ejercitadas contra el esquema real); frontend implementado en `configuracion-cuenta.component.ts` (autoservicio, con advertencia explícita de irreversibilidad) y `users.component.ts` (admin)
- Estado: verificado

REQ-NF-019 — Ante un webhook de PayPal que llega duplicado, o un pedido con fondos ya retenidos en escrow que necesita cancelarse, el sistema debe tener un comportamiento determinista y auditable: idempotencia ante reintentos del mismo webhook, un mecanismo de reconciliación (consulta activa a la API de PayPal) si el webhook no llega en una ventana razonable, y un flujo de reembolso o liberación explícito ante cancelación con fondos retenidos.

- Rationale: con patrón escrow, la ausencia de un flujo de cancelación-con-fondos-retenidos es el riesgo financiero más serio del corpus: hoy, un pedido con dinero en escrow no tiene camino de cancelación.
- Prioridad: Must
- Aceptación: un reintento del mismo webhook de PayPal (mismo pago, mismo estado de fondos) no genera una segunda actualización de estado ni un segundo registro de transacción; un pedido cancelado con fondos en escrow dispara un flujo de reembolso o retención documentado; existe un job o endpoint de reconciliación que consulta el estado real en PayPal para pedidos con webhook pendiente más allá de un umbral de tiempo configurable.
- Verificación: Test (`PaymentServiceImplWebhookTest.reintentoNoDuplica` cubre la idempotencia por estado de fondos; `PaymentServiceImplCancellationTest` cubre el reembolso/liberación; `PayPalReconciliationSchedulerTest` + `PayPalReconciliationExecutorServiceTest` cubren la reconciliación)
- Estado: implementado (`PayPalReconciliationScheduler/PayPalReconciliationExecutorService` nuevos, umbral configurable vía `paypal.reconciliacion.umbral-minutos`; `PaymentServiceImpl.cancelarPedidoConFondosRetenidos` nuevo, con reembolso vía PayPal o liberación solo-admin; guard añadido en `DeliverableServiceImpl.aprobarEntrega` para que un pedido ya cancelado no pueda aprobarse y pagarse dos veces; falta una prueba contra el sandbox real de PayPal para subir a verificado — ver excepciones-estado.txt)

REQ-NF-020 — Los contratos firmados deben conservarse durante un período declarado, con integridad verificable del hash de firma.

- Rationale: `generarHashFirma()` (`ContractServiceImpl`) calcula el hash una sola vez al firmar; nada lo recompute ni lo re-verifica después, y la tabla `contratos` no declara retención ni expiración — un contrato es evidencia legal del acuerdo entre las partes y no puede depender de que nadie lo borre por accidente ni de que el hash nunca se corrompa sin detectarlo.
- Prioridad: Should

- Aceptación: existe un período de retención declarado para los contratos firmados; existe un mecanismo (manual o automatizado) que re-verifica el hash de firma contra el contenido del contrato y señala una discrepancia si el hash no coincide.
- Verificación: Test (`ContractServiceImplTest` — congela contenido y hash al completarse la segunda firma, detecta discrepancia por manipulación posterior; `ContractIntegritySchedulerTest`; `ContractIntegrityControllerTest`)
- Estado: verificado (se introdujo un hash de CONTENIDO nuevo y separado del hash de firma existente — este último es una huella evento/quién/cuándo con `Instant.now()`, nunca reproducible, y se deja intacto con ese propósito; el nuevo hash se calcula una sola vez sobre el HTML ya renderizado y congelado al completarse la segunda firma. `ContractIntegrityScheduler` reverifica todos los contratos firmados cada noche y `POST /api/v1/admin/contratos/{id}/verificar-integridad` permite una verificación puntual. Retención declarada vía `contrato.retencion-anios` — confirmar la cifra exacta bajo la normativa ecuatoriana aplicable)

REQ-NF-021 — Las contraseñas deben cumplir una política de complejidad mínima (longitud y composición), y cambiar la contraseña debe revocar las demás sesiones activas del usuario.

- Rationale: una contraseña débil es el vector de compromiso de cuenta más común; revocar sesiones activas al cambiarla cierra la ventana en la que un atacante con la contraseña anterior podría seguir usando una sesión ya iniciada.
- Prioridad: Should
- Aceptación: una contraseña de menos de 8 caracteres, o sin al menos un dígito, una minúscula y una mayúscula, es rechazada al registrarse; cambiar la contraseña invalida cualquier sesión activa distinta de la que originó el cambio.
- Verificación: Test (`RegisterRequest` valida `@Size(min=8,max=100)` + `@Pattern` con dígito/minúscula/mayúscula obligatorios; `UserServiceImplTest.changePassword_ShouldUpdateHash_WhenContraseñaA` verifica explícitamente `sessionRevocationService.revocarSesionesUsuario(...)` tras el cambio)
- Estado: verificado

REQ-NF-022 — Los endpoints de autenticación y registro deben limitar la tasa de solicitudes por origen para mitigar fuerza bruta y abuso.

- Rationale: sin límite de tasa, un endpoint de login o registro es trivialmente atacable por fuerza bruta o creación masiva de cuentas; REQ-NF-015 ya menciona la infraestructura de conteo de intentos, pero ningún requisito fija los valores concretos que aplican hoy.
- Prioridad: Must
- Aceptación: login y verificación 2FA admiten como máximo 10 solicitudes por 60 segundos por origen; recuperar contraseña admite 5 solicitudes por 15 minutos; resetear contraseña admite 10 solicitudes por 15 minutos; registro admite 5 solicitudes por 60 minutos; superar el límite responde con un rechazo explícito, no con una degradación silenciosa.
- Verificación: Test (`AuthRateLimitFilterTest`, con los 5 límites cubiertos en su valor exacto y límite+1) + inspección de código (`AuthRateLimitFilter`, constantes hardcodeadas: login 10/60s, 2FA 10/60s, forgot-password 5/15min, reset-password 10/15min, registro 5/60min)
- Estado: verificado

REQ-NF-023 — La interfaz debe apuntar a **WCAG 2.2 Nivel AA** como estándar de referencia de accesibilidad, verificado en la medida en que la auditoría automatizada de Lighthouse lo permite.

- Rationale: la accesibilidad no es opcional para una plataforma pública que conecta clientes y creadores; sin un estándar de referencia declarado y un umbral medible, una regresión de accesibilidad no tiene ninguna alarma que la detecte. Se nombra WCAG 2.2 AA explícitamente porque es el estándar que Lighthouse usa como base para su categoría Accessibility — declararlo evita que un puntaje de Lighthouse se lea como un sinónimo de conformidad WCAG cuando no lo es.
- Prioridad: Should
- Aceptación: la categoría Accessibility de Lighthouse alcanza un puntaje $\geq 90/100$ sobre las vistas principales del catálogo, perfil y flujo de contratación.
- Verificación: docs/mediciones/lighthouse/REPORTE-LIGHTHOUSE.md. Alcance real de esta veri-

ficación, declarado sin ambigüedad: Lighthouse audita automáticamente un subconjunto de los criterios de éxito de WCAG 2.2 (los que admiten comprobación programática — contraste de color, atributos ARIA presentes, etiquetas de formulario, orden del DOM, etc.); un puntaje de 100/100 en Lighthouse **no equivale a una certificación de conformidad WCAG 2.2 AA completa**. Quedan explícitamente sin verificar: navegación completa por teclado de principio a fin en los flujos críticos (catálogo, contratación, pagos), uso real con lector de pantalla (NVDA/VoiceOver/JAWS), y los criterios de éxito de WCAG que dependen de juicio humano (orden de foco lógico, textos alternativos con sentido contextual, mensajes de error comprensibles). Ninguna de estas verificaciones manuales se ejecutó en este proyecto.

- Estado: verificado (100/100 en la medición más reciente, tras una regresión intermedia a 87-89/100 contra producción ya remediada de vuelta a 100/100) — **verificado únicamente contra el subconjunto automatizable de WCAG 2.2 AA que cubre Lighthouse; no constituye una auditoría de conformidad WCAG 2.2 AA completa.**

REQ-NF-024 — La base de datos debe contar con una política de respaldo y recuperación, con frecuencia y retención declaradas.

- Rationale: sin respaldo automatizado, la pérdida o corrupción de la base de datos de producción sería irrecuperable; los volúmenes de `docker-compose.yml` son de persistencia normal, no de respaldo, y los dumps SQL existentes en el repositorio son manuales y ad hoc, no una política.
- Prioridad: Should
- Aceptación: existe un mecanismo de respaldo automatizado (cron o equivalente) con frecuencia y retención declaradas; existe al menos una restauración de prueba documentada.
- Verificación: inspección de infraestructura + demostración de restauración
- Estado: verificado (mecanismo de respaldo automatizado FULL/INCREMENTAL con frecuencia y retención declaradas por programación cron o disparo manual, desde el panel de administración; restauración de prueba ejecutada y documentada el 2026-09-10 — ver `docs/despliegue/BACKUP.md` §Registro de restauraciones de prueba)

REQ-NF-025 — El usuario debe poder revocar todas sus propias sesiones activas ante sospecha de compromiso de su cuenta, sin depender de un Administrador.

- Rationale: REQ-F-030 solo cubre la revocación de sesiones de un tercero por un Administrador; sin esta capacidad, un usuario que sospecha que su cuenta fue comprometida no tiene forma de cerrar sus propias sesiones activas sin escalar a soporte.
- Prioridad: Should
- Aceptación: un usuario autenticado puede revocar todas sus sesiones activas con una sola acción; la sesión que originó la revocación puede, según diseño, cerrarse también o mantenerse — el comportamiento elegido queda documentado.
- Verificación: Test (`UserServiceImplTest.revokeAllMySessions_ShouldRevoke`, `UserControllerTest.revokeAllMySessions_ShouldRevoke` — DELETE /api/v1/usuarios/me/sesiones)
- Estado: verificado

5. Matriz de trazabilidad (resumen)

Ver archivo completo en `docs/trazabilidad/matriz.csv`. Estructura de columnas: `id_requisito`, `tipo`, `prioridad_moscow`, `historia_usuario`, `caso_de_uso`, `modulo_codigo`, `endpoint_api`, `prueba_automatizada`, `tipo_acceso`, `evidencia_empirica`, `estado`.

6. Evolución de requisitos desde la Entrega 1A

Cambio	Detalle
Renombrado de IDs	RF-NN → REQ-F-ONN, RNF-NN → REQ-NF-ONN, sin alterar el contenido semántico, para conformidad con ISO/IEC/IEEE 29148.
Adición de atributos	Se agregó rationale , estado (pendiente/implementado/verificado) y método de verificación explícito a cada requisito, ausentes en la tabla original de la Entrega 1A.
Resolución de brechas	REQ-F-009, 014, 015, 016, 023 (módulos social, comunicación, WebSockets) implementados y marcados como verificado para v1.0.0. REQ-F-010 (comentarios de portafolio) pasó de pendiente a implementado : se construyó la rodaja vertical completa (servicio, controlador y moderación admin) con borrado lógico, ya sin necesidad de excepción por ser Should. Queda pendiente su prueba automatizada para subirlo a verificado .
Resolución de brechas	REQ-NF-004, 007, 008, 011 resueltos con el frontend Angular 22 finalizado y la integración de Azure Blob Storage.
Módulo de Auditoría	REQ-NF-013 implementado formalmente mediante la migración V12, funciones PL/pgSQL, y aspecto AOP (@Auditable).
Reescritura de enunciado (v1.1.0)	REQ-F-016 y REQ-F-017 cambiaron de enunciado (ver CHANGELOG-REQ.md v1.1.0): el cuestionario (briefing) pasó de ser un formulario genérico enviado manualmente por el Creador tras crear el pedido, a estar ligado a un servicio concreto y ser obligatorio dentro del mismo formulario de creación del pedido; la plantilla de contrato pasó de ser una única plantilla global sembrada por migración, a un catálogo de plantillas curado por Administrador que el Creador elige por servicio.

Todo cambio adicional debe registrarse en `docs/requisitos/CHANGELOG-REQ.md` con fecha, autor, requisito afectado y motivo, siguiendo la convención Keep a Changelog.

7. Métricas de calidad del corpus de requisitos

Todas las cifras se derivan de `docs/trazabilidad/matriz.csv` en la fecha de este documento y son reproducibles ejecutando `scripts/validate-traceability.sh`, que además impide que estas métricas se desincronicen del SRS.

7.1 Volumen y distribución

Métrica	Valor
Total de requisitos	62
Por tipo	35 funcionales (56,5 %) · 27 no funcionales (43,5 %)
Por prioridad MoSCoW	37 Must (59,7 %) · 23 Should (37,1 %) · 2 Could (3,2 %)
Por estrategia de acceso a datos	43 CRUD-ORM · 8 SP · 11 sin acceso a datos (frontend/arquitectura/pendiente)

El corpus original de la Entrega 1A (37 requisitos, REQ-F-001 a REQ-F-023 y REQ-NF-001 a REQ-NF-014) se amplió en v1.1.2 con 11 requisitos adicionales (REQ-F-024 a REQ-F-031, REQ-NF-015 a REQ-NF-017) que documentan funcionalidad ya implementada, y en v1.3.0 con 10 requisitos adicionales más (REQ-F-032, REQ-F-033, REQ-NF-018 a REQ-NF-025) que documentan alcance genuinamente nuevo o brechas de cumplimiento con implicación legal, financiera o de seguridad no especificadas antes — ver §3.1, §4.2 y `CHANGELOG-REQ.md` v1.3.0. Además, en v1.3.0 dos requisitos heredados de la Entrega 1A (REQ-F-022 y REQ-NF-001) se dividieron en sub-requisitos atómicos (REQ-F-022a/b/c, REQ-NF-001a/b/c) para que cada capacidad testable tenga su propio estado sin depender de leer una nota aparte; esa división no representa alcance nuevo — ver §7.4.

7.2 Estado de verificación

Estado	Requisitos	Porcentaje
verificado	53	85,5 %
implementado	9	14,5 %
pendiente	0	0 %

Desglose por prioridad, que es lo que evalúa el criterio D0R:

Prioridad	Verificado	Implementado	Pendiente	Cumple el mínimo exigido
Must	35 (94,6 %)	2	0	35 de 37; 2 con excepción declarada
Should	16	7	0	23 de 23; 0 con excepción obligatoria, 3 declarados por honestidad sin exigirlo
Could	2	0	0	Sin mínimo exigible

Los dos requisitos Must que no alcanzan **verificado** están declarados uno a uno, con su motivo y su condición de cierre, en `docs/trazabilidad/excepciones-estado.txt` — y agrupan dos situaciones distintas que conviene no tratar como equivalentes:

- **Falla confirmada en el entorno local, alcance incierto en producción** (REQ-NF-009): la política `restart: unless-stopped` y el healthcheck están correctamente declarados, pero la demostración real ejecutada el 2026-09-10 (`docker kill` sobre `pfc_backend` y `pfc_postgres`, Docker Desktop/WSL2 local) mostró que el reinicio automático **no se disparó** en ninguno de los dos casos — ver `docs/mediciones/resiliencia/REPORTE-RECUPERACION.md`. No es evidencia pendiente de algo que funciona: es un resultado negativo real. No se puede extrapolar a Render, que no usa `docker-compose` ni esta política para gestionar sus propios servicios.
- **Implementado y probado unitariamente, pendiente de evidencia contra el entorno real** (REQ-NF-019): la idempotencia por estado ante webhook duplicado, la reconciliación activa contra la API de PayPal (`PayPalReconciliationScheduler/PayPalReconciliationExecutorService`) y el flujo de reembolso/liberación ante cancelación con fondos en escrow (`PaymentServiceImpl.cancelarPedidoConFondosRet`), ya existen en código y están probados (`PaymentServiceImplWebhookTest`, `PaymentServiceImplCancellationTest`, `PayPalReconciliationSchedulerTest`, `PayPalReconciliationExecutorServiceTest`), pero contra respuestas de PayPal mockeadas, no contra el sandbox real. Al ser el requisito de mayor riesgo financiero del corpus (patrón escrow), no sube a **verificado** solo con prueba unitaria — falta ejercitar el flujo completo contra `api-m.sandbox.paypal.com` y archivar la evidencia.

REQ-NF-001a, REQ-NF-001b y REQ-NF-001c ya no figuran en este grupo: `scripts/verificar-tls-ssllabs.sh` fue registrado como `prueba_automatizada` en `matriz.csv` (el validador exige cadena no vacía, no una clase JUnit), y el análisis SSL Labs archivado el 2026-09-10 contra `artisync-frontend.onrender.com` (grade A+, solo TLS 1.2/1.3 aceptados, redirección HTTPS forzada confirmada — ver `docs/mediciones/sec/ssl-labs/REPORTE-SSL-LA`) respalda el cierre. Los tres subieron a **verificado**.

Los tres requisitos Should que quedaban en **pendiente** con excepción declarada se resolvieron el 2026-09-10. REQ-F-022b (`RevisionTicketPaymentServiceImpl` genera automáticamente la orden PayPal de un ticket de revisión que supera el límite configurado, con una entidad propia `pagos_ticket_revision` para no competir con el escrow principal) y REQ-F-022c (`RevisionTicketExpirationScheduler` rechaza automáticamente un ticket con cargo pendiente de pago tras 48h configurables) subieron a **implementado** — ya cumplen su mínimo formal sin excepción obligatoria, y se mantienen declarados en `excepciones-estado.txt` solo por honestidad porque su prueba contra el sandbox real de PayPal sigue pendiente. REQ-NF-020 subió a **verificado**: se introdujo un hash de contenido nuevo y separado del hash de firma existente (que se deja intacto como huella evento/quién/cuándo con `Instant.now()`, nunca reproducible por diseño), calculado una sola vez sobre el HTML ya renderizado y congelado al completarse la segunda firma, con reverificación nocturna (`ContractIntegrityScheduler`) y bajo

demanda (POST /api/v1/admin/contratos/{id}/verificar-integridad). Un requisito Should en **implementado** se documenta también por honestidad aunque ya cumple su mínimo formal (REQ-NF-017) — el resultado medido de usabilidad, 61,25/100, no alcanza el umbral propio del proyecto. REQ-F-010, REQ-F-033, REQ-NF-018 y REQ-NF-022 ya salieron de este grupo tras completar su prueba automatizada (AdminCommentControllerTest, ViolationServiceImplTest ampliado, PrivacidadServiceImplIT, AuthRateLimitFilterTest ampliado, respectivamente) y subieron a **verificado**. REQ-NF-005 y REQ-NF-006 salieron de este grupo el 2026-09-10 con pruebas automatizadas reales (ChatWebSocketLoadIT, ContratoPdfTimingIT — ver docs/mediciones/ws/REPORTE-WS.md y docs/mediciones/perf/REPORTE-PDF-CONTRATO.md) y subieron a **verificado**; la primera, además, expuso y forzó a corregir un defecto real de producción en el envío de mensajes de chat por WebSocket (WebSocketConfig no resolvía @AuthenticationPrincipal en mensajes STOMP). REQ-NF-024 salió de este grupo el 2026-09-10 tras ejecutar y documentar una restauración de prueba real contra el mecanismo automatizado del panel admin (respaldo FULL pg_restore exit 0, pedidos=4/usuarios=10/contratos=3 — ver docs/despliegue/BACKUP.md \$Registro de restauraciones de prueba) y sube a **verificado**.

7.3 Cobertura de trazabilidad

Métrica	Valor
Requisitos presentes en la matriz	62 / 62 (100 %)
Requisitos con prueba automatizada asociada	53 (85,5 %)
Requisitos Must con prueba automatizada	36 / 37 (97,3 %)
Requisitos con evidencia empírica archivada	40 (64,5 %)

El criterio D0R exige prueba automatizada para todo **Must** en estado **verificado**: el validador lo impone y hace fallar el pipeline si se incumple. Para **Should/Could**, **verificado** también admite sostenerse en evidencia empírica archivada sin una clase de prueba dedicada cuando la naturaleza de la medición lo justifica — por ejemplo, REQ-NF-016 y REQ-NF-023 se apoyan en reportes Ja-CoCo/Lighthouse, no en una clase de test. De los 2 requisitos **Must** que no alcanzan **verificado**, REQ-NF-009 depende de una demostración operativa (caída/recuperación de contenedor) que por su propia naturaleza no se ejecuta como una clase de prueba del repositorio; REQ-NF-019 sí tiene prueba automatizada (PaymentServiceImplWebhookTest, PaymentServiceImplCancellationTest, PayPalReconciliationSchedulerTest, PayPalReconciliationExecutorServiceTest), pero solo contra respuestas de PayPal mockeadas — al ser el requisito de mayor riesgo financiero del corpus, no sube a **verificado** sin evidencia contra el sandbox real. REQ-NF-001a, REQ-NF-001b y REQ-NF-001c ya no figuran aquí: scripts/verificar-tls-ssllabs.sh fue registrado como **prueba_automatizada** (el validador solo exige cadena no vacía; el script tiene asserts y exit code 0/1), cerrando los tres sub-requisitos como **verificado**. REQ-F-010, REQ-F-033, REQ-NF-005, REQ-NF-006, REQ-NF-011, REQ-NF-018, REQ-NF-020 y REQ-NF-022 también alcanzaron **verificado** en rondas anteriores.

Nota sobre REQ-NF-011 y esta cifra. Al fijar AZURE_STORAGE_CONNECTION_STRING en Render y subir REQ-NF-011 a **verificado**, la columna **evidencia_empirica** de su fila en **matriz.csv** se actualizó de una nota de evidencia parcial (docs/mediciones/sec/..., que documentaba el incumplimiento) a la evidencia real del cierre: docs/despliegue/EVIDENCIA-AZURE.md, que registra la respuesta 302 de la API hacia una URL firmada *.blob.core.windows.net y la descarga 200 OK posterior directamente desde Azure Blob Storage. Este cambio de NF-011 en particular no altera el total por sí solo — es un swap de evidencia, no una incorporación —; el total de 40 en esta ronda refleja además la incorporación de REQ-NF-019, REQ-F-022b, REQ-F-022c y REQ-NF-020, cada uno respaldado por el mismo estándar de evidencia archivada que el resto (SSL Labs, REPORTE-RECUPERACION.md, BACKUP.md).

7.4 Estabilidad de requisitos

La tasa de estabilidad se calcula como $1 - (\text{requisitos modificados} / \text{requisitos totales})$ entre la Entrega 1A y la Entrega Final, tomando como modificación cualquier cambio de **enunciado**, **prioridad**

o **alcance** registrado en `CHANGELOG-REQ.md`. No cuentan los cambios de estado, que reflejan el avance de la implementación y no inestabilidad de la especificación. Dividir un requisito heredado en sub-requisitos atómicos (REQ-F-022, REQ-NF-001 en v1.3.0) sí cuenta como modificación de enunciado sobre el corpus heredado, aunque no añada alcance: el contenido testable es el mismo, pero la forma de expresarlo cambió.

Métrica	Valor
Requisitos en la Entrega 1A	37 (RF-01 a RF-23 · RNF-01 a RNF-14)
Requisitos en v1.2.0	48 (+11 añadidos sobre el corpus heredado)
Requisitos en v1.3.0	62 (+10 añadidos; +4 filas por dividir 2 requisitos heredados en sub-requisitos atómicos)
Añadidos (acumulado desde 1A)	21 (11 en v1.2.0 + 10 en v1.3.0)
Eliminados	0
Modificados en enunciado o alcance	5 (REQ-F-016, REQ-F-017 en v1.1.0; REQ-F-022, REQ-NF-001 divididos en v1.3.0; REQ-F-003 en v1.3.0)
Tasa de estabilidad del corpus heredado	$1 - 5/37 = 0,865$ (86,5 %)
Tasa de adición (acumulada)	$21/37 = 56,8$ % sobre el corpus original
Tasa de eliminación	0 %

El corpus heredado de la Entrega 1A permaneció **estable en volumen**: los mismos 37 requisitos originales, con correspondencia uno a uno de identificadores; ninguno se eliminó. Lo que cambió entre 1A y v1.0.0 fue la *forma* de la especificación, no su contenido: la reenumeración de RF-NN/RNF-NN a REQ-F-NNN/REQ-NF-NNN para conformidad con ISO/IEC/IEEE 29148, y el enriquecimiento de cada requisito con rationale, criterio de aceptación medible, método de verificación y estado — esos cambios no alteran lo que el sistema debe hacer y no se contabilizan como modificaciones. Entre v1.0.0 y v1.1.0 hubo dos cambios sustantivos de enunciado sobre el corpus heredado: REQ-F-016 (cuestionario ligado al servicio y obligatorio al crear el pedido, en vez de envío manual posterior) y REQ-F-017 (catálogo de plantillas de contrato curado por Administrador, en vez de una plantilla global única). En v1.3.0 hubo dos modificaciones más, de naturaleza distinta: REQ-F-022 y REQ-NF-001 no cambiaron de significado, pero se dividieron cada uno en tres sub-requisitos atómicos (a/b/c) porque agrupaban capacidades con estados de verificación distintos bajo un único identificador — una de esas capacidades no debía quedar oculta detrás del estado de las otras dos. Las cuatro modificaciones están documentadas con su motivo en `CHANGELOG-REQ.md` (v1.1.0 y v1.3.0 respectivamente). En v1.3.0 se añade una quinta: REQ-F-003, cuyo enunciado se amplió para declarar el refresh token de 7 días (antes solo mencionado en el criterio de aceptación). Por separado, se incorporaron 21 requisitos nuevos en total (11 en v1.2.0, 10 en v1.3.0) que no son alcance nuevo del sistema construido, sino especificación de funcionalidad y brechas que ya existían en el código o en la operación real, pero que ningún requisito capturaba — ver §3.1, §4.2 y `CHANGELOG-REQ.md`. La tasa de estabilidad de 86,5% se calcula solo sobre el corpus heredado (denominador 37), porque mide cuánto cambió el *enunciado* de lo ya especificado; los 21 requisitos nuevos se reportan aparte como tasa de adición, no como inestabilidad, porque documentan alcance que nunca había sido especificado, no un enunciado que cambió de significado.

Conviene leer estas cifras con cautela metodológica: que el corpus heredado cambiara relativamente poco de enunciado (89,2% de estabilidad) mientras el corpus total más que se duplicó ($37 \rightarrow 62$, +67,6 %) a lo largo de tres rondas de auditoría es coherente con un proyecto académico de alcance cerrado, donde la especificación original se congeló temprano y la brecha entre “lo que se construyó” y “lo que se documentó” se fue cerrando en rondas sucesivas de revisión externa, no durante el desarrollo original. En un proyecto con stakeholders externos, una tasa de adición acumulada de esta magnitud sería una señal de alarma sobre el proceso de especificación continua, no solo un ajuste de documentación. La limitación se declara en el capítulo de amenazas a la validez del documento académico.

8. Aprobación

Este SRS se somete a la revisión y aprobación del docente-director del PFC, conforme al apartado A.3.1 de la guía de la Entrega Final.

Rol	Nombre	Fecha	Firma
Docente-director del PFC	Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.		
Representante del equipo	Johan Stalin Carvajal Loor		

Estado de la aprobación: pendiente de firma. La firma depende de la disponibilidad de un tercero externo al equipo (el docente-director) y no puede completarse unilateralmente antes de la entrega. Dado que la Entrega Final se presenta durante la semana del examen final (semana 19, 7–11 de septiembre de 2026), la revisión y, de proceder, la formalización de esta firma se realizarán presencialmente **el día del examen**, que es la primera instancia en que ambas partes coinciden. Hasta que esta sección lleve la firma del docente-director, el criterio D0R no puede superar el nivel *En desarrollo*, según la regla transversal 9 de la guía. La versión aprobada y firmada, cuando exista, se archiva como `docs/requisitos/SRS-v1.3.0.pdf`; las versiones anteriores (incluidas v1.0.0 y v1.2.0) se conservan en `docs/requisitos/historico/`.